

**Assignment 4, Part 2: Practical Exploration of Instruction-Level Parallelism Techniques in
gem5**

Sriman Cherukuru

University of the Cumberland

Computer Architecture

Assignment 4, Part 2: gem5 Experiments

September 13, 2026

Assignment 4, Part 2: Practical Exploration of Instruction-Level Parallelism Techniques in gem5

This report presents the hands-on half of Assignment 4. Using the gem5 simulator already built for Assignments 1 and 2 (gem5 v25.1.0.1, X86, syscall-emulation mode), I configured a pipeline, ran seven small self-checking programs through it, and measured how four ILP techniques change its behavior: the basic pipeline itself, branch prediction, multiple issue, and simultaneous multithreading (SMT). In total 98 simulations were run from one parameterized configuration script. Two small additions were made to gem5 itself: a static branch predictor (StaticBP), because gem5 ships no “no prediction” option, and a fix for a crash in its SMT load/store-queue code. Everything (configuration, workloads, scripts, raw statistics, charts, screenshots and this report) is in the GitHub repository <https://github.com/srimancherukuru/gem5-ilp-assignment4>.

Environment and Method

Simulator, Host and Toolchain

The simulator is the gem5 stable release v25.1.0.1 cloned in Assignment 1 and built as build/X86/gem5.opt with `scons -j 16` on a MacBook with an Apple M5 Max (18 cores, 48 GB) running macOS 26.4.1, clang 16 and Python 3.14. Because macOS has no x86-64 Linux GCC, the guest programs are cross-compiled with `zig cc` to statically linked x86-64 musl ELF executables, the same recipe that worked for the Hello World program in Assignment 2. Vectorization and loop unrolling are disabled so that the kernels stay scalar integer and scalar double-precision code whose instruction classes are easy to read in the statistics (Figure 1).

Figure 1

Cross-Compiling the Seven Workloads With zig for gem5's x86-64 Syscall-Emulation Mode

```

workloads — -zsh — 118x26

srmancherukuru@Sriman-macbook gem5-ilp-assignment4 % cd workloads && make -B && ls -la ../bin | cut -c1-90
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops int_alu.c -o ../bin/i
nt_alu
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops dep_chain.c -o ../bin
/dep_chain
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops fp_matmul.c -o ../bin
/fp_matmul
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops mem_stream.c -o ../bi
n/mem_stream
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops mem_chase.c -o ../bin
/mem_chase
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops branchy.c -o ../bin/b
ranchy
zig cc -target x86_64-linux-musl -O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops bsort.c -o ../bin/bs
ort
total 17816
drwxr-xr-x  9 srmancherukuru  staff    288 Sep 13 13:36 .
drwxr-xr-x 12 srmancherukuru  staff    384 Sep 13 13:32 ..
-rwxr-xr-x  1 srmancherukuru  staff 1300144 Sep 13 13:36 branchy
-rwxr-xr-x  1 srmancherukuru  staff 1302896 Sep 13 13:36 bsort
-rwxr-xr-x  1 srmancherukuru  staff 1298896 Sep 13 13:36 dep_chain
-rwxr-xr-x  1 srmancherukuru  staff 1299864 Sep 13 13:36 fp_matmul
-rwxr-xr-x  1 srmancherukuru  staff 1301120 Sep 13 13:36 int_alu
-rwxr-xr-x  1 srmancherukuru  staff 1302192 Sep 13 13:36 mem_chase
-rwxr-xr-x  1 srmancherukuru  staff 1302280 Sep 13 13:36 mem_stream
srmancherukuru@Sriman-macbook workloads %

```

Note. The Makefile passes `-O2 -static -fno-vectorize -fno-slp-vectorize -fno-unroll-loops`; each binary is a self-contained 1.3 MB static ELF file.

The Configuration Script

One script, `configs/ilp_config.py`, drives every experiment. It builds a two-level cache hierarchy that never changes (32 KiB 4-way instruction cache and 64 KiB 4-way data cache with 2-cycle latency, a 512 KiB 8-way L2 with 10-cycle latency, a crossbar and a DDR3-1600 channel with 1 GiB) and then instantiates the core the command line asks for. The out-of-order core is gem5's X86O3CPU. Its issue width sets the fetch, decode, rename, dispatch, issue, write-back and commit widths together, and the reorder buffer (ROB), instruction queue (IQ), load and store queues and functional-unit pool are scaled with it (Table 1). The direction predictor is selectable, the branch target buffer (BTB), return address stack and indirect predictor stay at gem5's defaults, and giving several `--cmd` programs turns the core into an SMT core with one hardware thread per program. Figures 2 to 4 show the three parts of the script; the complete listing is Appendix A.

Table 1
Out-of-Order Core Configurations Used in the Issue-Width Sweep

Parameter	1-wide	2-wide	4-wide	8-wide
Fetch/decode/rename/dispatch/issue/writeback/commit width	1	2	4	8
Reorder buffer entries	32	64	128	192
Instruction queue entries	16	32	64	96
Load queue / store queue entries	8 / 8	16 / 16	32 / 32	48 / 48
Integer ALUs	1	2	4	8
Integer multiply/divide, FP add, FP multiply/divide, SIMD units (each)	1	1	2	4
Load/store ports	1	1	2	4
Physical integer / FP registers (gem5 default)	256 / 256	256 / 256	256 / 256	256 / 256
Direction predictor (default)	Tournament	Tournament	Tournament	Tournament
BTB / return address stack (default)	4096 entries, direct-mapped / 16	same	same	same
Clock; L1I / L1D; L2; DRAM	2 GHz; 32 KiB / 64 KiB 4-way; 512 KiB 8-way; DDR3-1600	same	same	same

Note. The 1-wide configuration is the “basic pipeline” of Section 2; widths 2, 4 and 8 are the superscalar configurations of Section 5. Per-thread partitions apply when SMT is enabled (Section 6).

Figure 2

ilp_config.py, Part 1: Cache Classes, Branch Predictor Selection and the Width-Scaled Functional-Unit Pool

```

gem5-ilp-assignment4 --zsh --110x52

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % sed -n '/^# -> branch predictors/,/^SIZES = /p' configs/
ilp_config.py
# ----- branch predictors
def make_bp(kind):
    """Return a BranchPredictor unit whose *direction* predictor is 'kind'.
    The BTB, return-address stack and indirect predictor stay at gem5's
    defaults so only the direction-prediction policy changes."""
    if kind == "static-nt":
        cond = StaticBP(predictTaken=False) # always fall through
    elif kind == "static-t":
        cond = StaticBP(predictTaken=True) # always take
    elif kind == "local":
        cond = LocalBP(localPredictorSize=2048, localCtrBits=2) # 2-bit counters
    elif kind == "tournament":
        cond = TournamentBP() # local + global + chooser
    else:
        cond = TAGE() # state-of-the-art tagged geometric
    return BranchPredictor(conditionalBranchPred=cond)

# ----- functional units
# Functional-unit pool scaled with the issue width: 'w' integer ALUs, and
# about w/2 of each of the other unit types (never fewer than one).
IntMulDivCls = getattr(objs, "X86IntMultDiv", IntMultDiv)

def make_fu_pool(w):
    half = max(1, w // 2)
    return FUPool(FUList=[
        IntALU(count=w),
        IntMulDivCls(count=half),
        FP.ALU(count=half),
        FP.MultDiv(count=half),
        SIMD_Unit(count=half),
        RdWrPort(count=half), # load/store ports
        PredALU(count=1),
        Matrix_Unit(count=1),
        System_Unit(count=1),
    ])

# Buffer sizes that grow with the width (ROB, IQ, LQ/SQ entries).
SIZES = {1: (32, 16, 8), 2: (64, 32, 16), 4: (128, 64, 32), 8: (192, 96, 48)}
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Note. The static-nt and static-t choices instantiate the StaticBP class added to gem5 for this assignment.

Figure 3

ilp_config.py, Part 2: Building the Out-of-Order Core (and the In-Order MinorCPU Alternative)

```

gem5-ilp-assignment4 — -zsh — 110x46

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % sed -n '/^def make_o3/,/^# --* workloads/p' configs/ilp_c
onfig.py
def make_o3(nthreads):
    w = args.width
    rob, iq, lsq = SIZES[w]
    cpu = X86O3CPU(numThreads=nthreads)
    for p in ("fetchWidth", "decodeWidth", "renameWidth", "dispatchWidth",
            "issueWidth", "wbWidth", "commitWidth", "squashWidth"):
        setattr(cpu, p, w)
    cpu.numROBEntries = args.rob or rob
    cpu.instQueues = [IQUnit(numEntries=args.iq or iq)] # IQ lives in IQUnit in gem5 v25
    cpu.lqEntries = args.lsq or lsq
    cpu.sqEntries = args.lsq or lsq
    cpu.fuPool = make_fu_pool(w)
    cpu.branchPred = make_bp(args.bp)
    if nthreads > 1:
        # SMT policies
        cpu.smtFetchPolicy = args.smt_fetch
        cpu.smtROBPolicy = args.smt_share
        cpu.smtLSQPolicy = args.smt_share
        cpu.smtNumFetchingThreads = 1
    return cpu

def make_minor(nthreads):
    """Single-issue in-order pipeline: Fetch1 -> Fetch2 -> Decode -> Execute."""
    cpu = X86MinorCPU(numThreads=nthreads)
    cpu.branchPred = make_bp(args.bp)
    if args.minor_stock:
        return cpu
    cpu.fetch1FetchLimit = 1
    cpu.fetch2InputBufferSize = 2
    cpu.decodeInputWidth = 1
    cpu.decodeInputBufferSize = 3
    cpu.executeInputWidth = 1
    cpu.executeIssueLimit = 1
    cpu.executeMemoryIssueLimit = 1
    cpu.executeCommitLimit = 1
    cpu.executeMemoryCommitLimit = 1
    cpu.executeInputBufferSize = 7
    return cpu

# ----- workloads
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % █

```

Note. All seven stage widths are set from --width; ROB, IQ and LSQ sizes come from the SIZES table; SMT fetch and sharing policies apply when more than one program is given.

Figure 4

ilp_config.py, Part 3: Processes, System, Caches, Memory, Interrupt Controllers and the Simulation Loop

```

gem5-ilp-assignment4 --zsh -- 110x58

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % sed -n '/^# -*/ workloads/$p' configs/ilp_config.py
# ----- workloads
procs = []
for idx, spec in enumerate(args.cmd):
    parts = spec.split(":")
    binary = os.path.abspath(parts[0])
    procs.append(Process(pid=100 + idx, cmd=[binary] + parts[1:]))
nthreads = len(procs)

# ----- system
system = System()
system.clk_domain = SrcClockDomain(clock=args.clock, voltage_domain=VoltageDomain())
system.mem_mode = "timing"
system.mem_ranges = [AddrRange("1GiB")]
if nthreads > 1:
    system.multi_thread = True

system.cpu = make_o3(nthreads) if args.cpu == "o3" else make_minor(nthreads)
if args.maxinsts:
    system.cpu.max_insts_any_thread = args.maxinsts

# L1 caches on the core, a shared L2, then the memory bus and DDR3.
system.cpu.icache = L1ICache()
system.cpu.dcache = L1DCache()
system.cpu.icache.cpu_side = system.cpu.icache_port
system.cpu.dcache.cpu_side = system.cpu.dcache_port
system.l2bus = L2XBar()
system.cpu.icache.mem_side = system.l2bus.cpu_side_ports
system.cpu.dcache.mem_side = system.l2bus.cpu_side_ports
system.l2cache = L2Cache()
system.l2cache.cpu_side = system.l2bus.mem_side_ports
system.membus = SystemXBar()
system.l2cache.mem_side = system.membus.cpu_side_ports
system.mem_ctrl = MemCtrl()
system.mem_ctrl.dram = DDR3_1600_8x8(range=system.mem_ranges[0])
system.mem_ctrl.port = system.membus.mem_side_ports
system.system_port = system.membus.cpu_side_ports

# x86 needs one interrupt controller per hardware thread, each with 3 ports.
system.cpu.createInterruptController()
for i in range(nthreads):
    system.cpu.interrupts[i].pio = system.membus.mem_side_ports
    system.cpu.interrupts[i].int_requestor = system.membus.cpu_side_ports
    system.cpu.interrupts[i].int_responder = system.membus.mem_side_ports

system.workload = SEWorkload.init_compatible(procs[0].cmd[0])
system.cpu.workload = procs if nthreads > 1 else procs[0]
system.cpu.createThreads()

# ----- simulate
root = Root(full_system=False, system=system)
m5.instantiate()
print(f"[ilp_config] cpu={args.cpu} width={args.width} bp={args.bp} "
      f"threads={nthreads} cmds={args.cmd}")
exit_event = m5.simulate()
ticks = m5.curTick()
print(f"[ilp_config] exiting @ tick {ticks} because {exit_event.getCause()}")
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Note. Each `--cmd` becomes a `Process` with its own `PID`; the x86 core needs one interrupt controller per hardware thread, each wired with three ports.

Workloads

Seven small C programs were written so that each stresses one kind of instruction-level behavior (Table 2). Each takes an iteration count, runs for three to seven million instructions, and prints a checksum, so every gem5 run can be validated against a native run of the same source (Figure 5): all seven checksums match, which confirms that the simulator executed the programs correctly. The sources are in Appendix C.

Table 2
The Seven Workloads

Program	What it stresses	Argument	Instructions	$\mu\text{ops} / \text{inst.}$	L1D miss rate
int_alu	Eight independent xorshift lanes: integer ALU work with high ILP	int_alu:50000	3,703,879	1.04	5.46%
dep_chain	One serial multiply-add chain: no ILP (the limit case)	dep_chain:500000	4,003,955	1.63	5.47%
fp_matmul	48×48 double-precision matrix multiply: FP multiply/add, L1-resident	fp_matmul:4	3,268,564	1.85	0.07%
mem_stream	Sequential sweep of a 4 MiB array: memory bandwidth/latency bound	mem_stream:2	6,819,943	2.08	70.21%
mem_chase	Pointer chase, one cache line per hop over 2 MiB: dependent misses	mem_chase:300000	4,349,775	1.81	40.29%
branchy	Random data-dependent branches (50/50 and 25/75): predictor stress	branchy:20	3,892,612	2.03	0.51%
bsort	Bubble sort of 800 random keys: mixed integer/memory/branches	bsort:800	2,581,636	1.75	0.02%

Note. Instruction counts, micro-op (μop) ratios and miss rates are from the 4-wide runs. x86 instructions decode into one or more μops ; gem5 reports IPC in instructions and the pipeline viewer records μops .

Figure 5
Validation: Native Outputs (Apple M-Series, clang) Versus gem5 Outputs (x86-64 Out-of-Order Core) for the Same Arguments

```

gem5-ilp-assignment4 — zsh — 118x22

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % echo '--- native (Apple M-series, clang -O2):'; cat results/native_checksums.txt; echo; echo '--- gem5 X86 O3 4-wide:'; grep -h -E '^(int_alu|dep_chain|fp_matmul|mem_stream|mem_chase|branchy|bsort):' results/runs/ss_w4_*/run.log
--- native (Apple M-series, clang -O2):
int_alu: n=50000 checksum=da8cf086078afda2
dep_chain: n=500000 checksum=cc679c89b22ae8b9
fp_matmul: reps=4 checksum=287734899
mem_stream: passes=2 checksum=8dd97a8432780000
mem_chase: hops=300000 final=168928
branchy: passes=20 checksum=0000a07e9043b778 hist=165100
bsort: n=800 sorted=1 checksum=9facde812add0c98

--- gem5 X86 O3 4-wide:
branchy: passes=20 checksum=0000a07e9043b778 hist=165100
bsort: n=800 sorted=1 checksum=9facde812add0c98
dep_chain: n=500000 checksum=cc679c89b22ae8b9
fp_matmul: reps=4 checksum=287734899
int_alu: n=50000 checksum=da8cf086078afda2
mem_chase: hops=300000 final=168928
mem_stream: passes=2 checksum=8dd97a8432780000
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Note. The order differs (the gem5 list is alphabetical) but every checksum is identical, so the simulated executions are correct.

Metrics and Tooling

Throughput is instructions per cycle (IPC), computed from the committed-instruction and cycle counters in each run's stats.txt; its inverse is CPI. Instruction latency is measured from gem5's O3PipeView trace, which records for every micro-op the tick at which it was fetched, decoded, renamed, dispatched, issued, completed and retired. A small Python script (pipestats.py) turns that into the mean, median and 90th-percentile fetch-to-commit latency, the average delay between consecutive stages, the fraction of fetched μ ops that were squashed, and a Gantt-style pipeline diagram. Branch behavior comes from the predictor's condPredicted/condIncorrect counters and the commit stage's misprediction counter (reported as a rate and as mispredictions per thousand instructions, MPKI), and structural pressure from the rename stage's ROB-full, IQ-full and store-queue-full event counters. All 98 runs were launched from generated run matrices, and parse_stats.py aggregates every stats.txt into results/summary.csv (Figure 6).

Figure 6

The Results Directory: One Folder per Simulation Run With stats.txt, config.ini and run.log

```

gem5-ilp-assignment4 — zsh — 118x26

pv_w1_dep_chain      ss_w2_branchy
pv_w1_fp_matmul      ss_w2_bsort
pv_w1_int_alu         ss_w2_dep_chain
pv_w1_mem_chase       ss_w2_fp_matmul
pv_w1_mem_stream      ss_w2_int_alu
pv_w2_branchy         ss_w2_mem_chase
pv_w2_bsort           ss_w2_mem_stream
pv_w2_dep_chain       ss_w4_branchy
pv_w2_fp_matmul       ss_w4_bsort
pv_w2_int_alu         ss_w4_dep_chain
pv_w2_mem_chase       ss_w4_fp_matmul
pv_w2_mem_stream      ss_w4_int_alu
pv_w4_branchy         ss_w4_mem_chase
pv_w4_bsort           ss_w4_mem_stream
pv_w4_dep_chain       ss_w8_branchy
pv_w4_fp_matmul       ss_w8_bsort
pv_w4_int_alu         ss_w8_dep_chain
pv_w4_mem_chase       ss_w8_fp_matmul
pv_w4_mem_stream      ss_w8_int_alu
pv_w8_branchy         ss_w8_mem_chase
pv_w8_bsort           ss_w8_mem_stream
pv_w8_dep_chain

citations.bib  config.ini  config.json  run.log      stats.txt
101 results/runs/_done.log
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Adding a Static Predictor to gem5

The assignment asks for the pipeline “with and without branch prediction”. gem5’s out-of-order core cannot run without a predictor object, and it ships only dynamic predictors (local, tournament, bi-mode, TAGE variants and perceptrons). I therefore added StaticBP, a ConditionalPredictor subclass modeled on LocalBP: lookup() returns a fixed direction (not-taken by default, taken when predictTaken is set) and update() learns nothing (Figures 7 to 9). The class was registered in the SConscript and in BranchPredictor.py, committed on a branch of the gem5 clone (Figure 10), and gem5 was rebuilt; only the new files and the generated parameter headers recompiled (Figure 11). Appendix B lists the source, and the patch is in the repository.

Figure 7

static_bp.hh: The StaticBP Class Declaration (Interface Identical to gem5’s LocalBP)

```

srincherukuru@Sriman-macbook gem5-llp-assignment4 % sed -n 12,54p ~/git/gem5/src/cpu/pred/static_bp.hh
#ifndef __CPU_PRED_STATIC_BP_HH__
#define __CPU_PRED_STATIC_BP_HH__

#include "base/types.hh"
#include "cpu/pred/conditional.hh"
#include "params/StaticBP.hh"

namespace gem5
{
namespace branch_prediction
{
class StaticBP : public ConditionalPredictor
{
public:
    StaticBP(const StaticBPParams &params);

    bool lookup(ThreadID tid, Addr pc, void * &bp_history) override;

    void branchPlaceholder(ThreadID tid, Addr pc, bool uncond,
        void * &bp_history) override;

    void updateHistories(ThreadID tid, Addr pc, bool uncond, bool taken,
        Addr target, const StaticInstPtr &inst,
        void * &bp_history) override;

    void update(ThreadID tid, Addr pc, bool taken,
        void * &bp_history, bool squashed,
        const StaticInstPtr &inst, Addr target) override;

    void squash(ThreadID tid, void * &bp_history) override
    { assert(bp_history == NULL); }

private:
    /** The one direction every conditional branch is predicted with. */
    const bool predictTaken;
};
} // namespace branch_prediction
} // namespace gem5

#endif // __CPU_PRED_STATIC_BP_HH__
srincherukuru@Sriman-macbook gem5-llp-assignment4 %

```

Figure 8

static_bp.cc: lookup() Returns the Fixed Direction; update() and the History Hooks Do Nothing

```

gem5-ilp-assignment4 --zsh --110x58

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % sed -n 1,56p ~/git/gem5/src/cpu/pred/static_bp.cc
/*
 * StaticBP -- static direction predictor (see static_bp.hh).
 */

#include "cpu/pred/static_bp.hh"

#include "base/trace.hh"
#include "debug/Fetch.hh"

namespace gem5
{
    namespace branch_prediction
    {
        StaticBP::StaticBP(const StaticBPParams &params)
            : ConditionalPredictor(params),
              predictTaken(params.predictTaken)
        {
            DPRINTF(Fetch, "StaticBP: every conditional branch is predicted %s\n",
                    predictTaken ? "taken" : "not-taken");
        }

        void
        StaticBP::branchPlaceholder(ThreadID tid, Addr pc, bool uncond,
                                    void * &bp_history)
        {
            // No predictor state, so there is no history object to create.
        }

        void
        StaticBP::updateHistories(ThreadID tid, Addr pc, bool uncond, bool taken,
                                   Addr target, const StaticInstPtr &inst,
                                   void * &bp_history)
        {
            // A static predictor keeps no history.
        }

        bool
        StaticBP::lookup(ThreadID tid, Addr pc, void * &bp_history)
        {
            DPRINTF(Fetch, "StaticBP: branch %x predicted %s\n", pc,
                    predictTaken ? "taken" : "not-taken");
            return predictTaken;
        }

        void
        StaticBP::update(ThreadID tid, Addr pc, bool taken, void * &bp_history,
                          bool squashed, const StaticInstPtr &inst, Addr target)
        {
            // Nothing is learned from the outcome: the prediction never changes.
            assert(bp_history == NULL);
        }
    } // namespace branch_prediction
} // namespace gem5
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Figure 9

The Python SimObject (BranchPredictor.py) With Its predictTaken Parameter, and the SConscript Registration

```

gem5-ilp-assignment4 --zsh --110x30

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -n -A15 'class StaticBP' ~/git/gem5/src/cpu/pred/Br
anchPredictor.py; grep -n 'StaticBP|static_bp' ~/git/gem5/src/cpu/pred/SConscript
267:class StaticBP(ConditionalPredictor):
268-    """Static (non-learning) direction predictor added for the ILP
269-    assignment: every conditional branch is predicted with one fixed
270-    direction, so the pipeline gets no dynamic branch prediction."""
271-
272-    type = "StaticBP"
273-    cxx_class = "gem5:branch_prediction:StaticBP"
274-    cxx_header = "cpu/pred/static_bp.hh"
275-
276-    predictTaken = Param.Bool(
277-        False,
278-        "Fixed direction for every conditional branch: False = always "
279-        "not-taken (fall through), True = always taken",
280-    )
281-
282-
52:    'StaticBP', 'TournamentBP', 'BiModeBP', 'TAGEBase', 'TAGE', 'LoopPredictor',
68:Source('static_bp.cc')
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Figure 10

The gem5 Working Copy: Branch ilp-assignment4 With the Two Commits Made for This Assignment on Top of v25.1.0.1

```

gem5 — -zsh — 118x22

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % cd ~/git/gem5 && git branch --show-current && git log --oneline
-3 && git diff stable --stat && ls -la build/X86/gem5.opt
ilp-assignment4
36df907415 (HEAD -> ilp-assignment4) cpu/pred: add StaticBP, a static always-(not-)taken direction predictor for the I
LP assignment
38716b67fb (origin/stable, stable) misc: add agentics-maintenance.yml to stable (#3459)
a28ae2f8b2 tests,github: rm unused `gh-aw` extension installation
.gitignore | 1 +
src/cpu/pred/BranchPredictor.py | 16 ++++++++
src/cpu/pred/SConscript | 4 +++-
src/cpu/pred/static_bp.cc | 56 ++++++++
src/cpu/pred/static_bp.hh | 54 ++++++++
src/sim/DVFSHandler.py | 12 ++++++++
src/sim/dvfs_handler.hh | 12 ++++++++
7 files changed, 153 insertions(+), 2 deletions(-)
-rwxr-xr-x 1 srimancherukuru staff 57783960 Sep 13 13:18 build/X86/gem5.opt
srimancherukuru@Sriman-macbook gem5 %

```

Figure 11

Incremental Rebuild: Only static_bp.cc and the Generated StaticBP Parameter Files Were Compiled

```

gem5-ilp-assignment4 — -zsh — 118x14

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -n -E 'static_bp|StaticBP|done building' results/logs/gem5_
rebuild_staticbp.log
533: [S0 Param] m5.objects.BranchPredictor, StaticBP -> X86/python/_m5/param_StaticBP.cc
557: [S0 Param] m5.objects.BranchPredictor, StaticBP -> X86/params/StaticBP.hh
876: [ CXX] src/cpu/pred/static_bp.cc -> X86/cpu/pred/static_bp.o
877: [ CXX] X86/python/_m5/param_StaticBP.cc -> .o
958:scons: done building targets.
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Basic Pipeline Simulation

gem5 has no textbook five-stage in-order model for x86 that works in this release (its in-order MinorCPU discards every wrong-path x86 instruction in a loop and needs more than 20 cycles per instruction; see Section 8), so the basic pipeline is the 1-wide out-of-order configuration of Table 1: one instruction per stage per cycle, a 32-entry window, one ALU and one memory port. Its stages are fetch, decode, rename, dispatch (into the instruction queue),

issue and execute (with the memory access for loads and stores), write-back (“complete”) and commit (“retire”). Two tiny workloads, `dep_chain` with 30 iterations and `int_alu` with 30 iterations, were run with the `O3PipeView` debug flag so that every μop of the run is traced.

Cycle-by-Cycle Behavior

Figure 12 shows the raw trace records for three consecutive instructions of the `dep_chain` loop: each record lists the μop and the tick (1 tick = 1 ps, 500 ticks = one 2 GHz cycle) at which it passed each stage. `gem5`’s pipeline viewer, `util/o3-pipeview.py`, renders these records as a timeline (Figure 13): `f`, `d`, `n`, `p`, `i`, `c` and `r` mark the cycle of fetch, decode, rename, dispatch, issue, completion and retirement, one row per μop in fetch order. The Fetch, Decode, Rename, IEW and Commit debug flags give the same information from the stages’ point of view, cycle by cycle (Figure 14). To make the flow easier to read I also drew Gantt charts from the trace with `matplotlib` (Figures 15 and 16): each row is a μop and each colored segment is the time it spent between two stages.

Figure 12

O3PipeView Trace Records for Three Consecutive Instructions of the dep_chain Loop (1-Wide Core)

```

gem5-ilp-assignment4 --zsh -- 118x30

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -n -A7 'O3PipeView:fetch:.*IMUL_R_R :' results/runs/pv_demo
_w1_dep_chain/trace.out | sed -n '25,48p' | cut -c1-110
22934-O3PipeView:decode:3668500
22935-O3PipeView:rename:3669000
22936-O3PipeView:dispatch:3670000
22937-O3PipeView:issue:3670000
22938-O3PipeView:complete:3671500
22939-O3PipeView:retire:3672500:store:0
22940-O3PipeView:fetch:3668000:0x01002170:1:3278: IMUL_R_R : mulel   rdx, rdx
22941-O3PipeView:decode:3669500
22942-O3PipeView:rename:3670000
22943-O3PipeView:dispatch:3671000
22944-O3PipeView:issue:3671500
22945-O3PipeView:complete:3672000
22946-O3PipeView:retire:3673000:store:0
22947-O3PipeView:fetch:3668500:0x01002170:2:3279: IMUL_R_R : muleh   t0, t0
22948-O3PipeView:decode:3670000
22949-O3PipeView:rename:3670500
22950-O3PipeView:dispatch:3671500
22951-O3PipeView:issue:3672000
22952-O3PipeView:complete:3672500
22953-O3PipeView:retire:3673500:store:0
22954-O3PipeView:fetch:3669000:0x01002174:0:3280: MOV_R_R : mov    r8, rdx, rdx
--
23024-O3PipeView:fetch:3675500:0x01002170:0:3290: IMUL_R_R : mulls   rdx, rax
23025-O3PipeView:decode:3676000
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Note. Each x86 instruction decodes into one or more μ ops (the IMUL into three: limm, mulls, mulel). The ticks are 500 apart, one per cycle; complete:0 marks a μ op that produced no result.

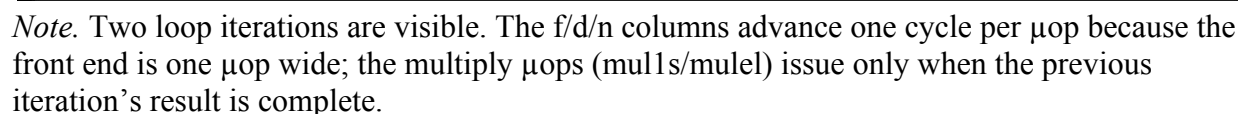


Figure 14

Stage-Level Debug Output (Fetch, Decode, Rename, IEW, Commit Flags) for Five Cycles of the Same Loop

```

gem5-ilp-assignment4 --zsh -- 118x48

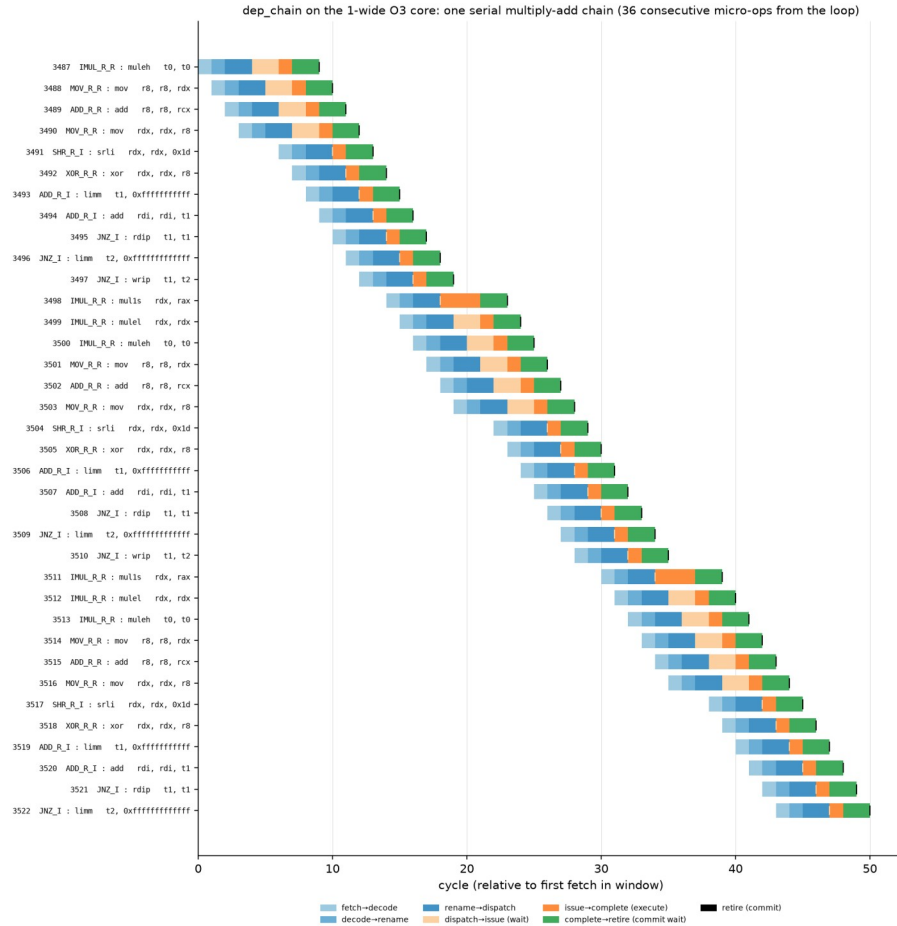
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % sed -n 1,44p results/logs/o3_stage_debug_dep_chain_w1.log
3470000: system.cpu.fetch: Running stage.
3470000: system.cpu.fetch: [tid:0] Attempting to fetch from
3470000: system.cpu.fetch: [tid:0] Adding instructions to queue to decode.
3470000: system.cpu.decoder: Decode: Decoded imul instruction:
{
    leg = 0,
    rex = 0x48,
    vex/xop = 0,
    op = {
        type = one byte,
        op = 0x6b,
    },
    modRM = 0xc0,
    sib = 0,
    immediate = 0xa,
    displacement = 0
    dispSize = 0}
3470000: system.cpu.fetch: [tid:0] Instruction PC (0x1005262=>0x1005266).(0=>1) created [sn:3123].
3470000: system.cpu.fetch: [tid:0] Instruction is:  IMUL_R_R_I : limm  t1, 0xa
3470000: system.cpu.fetch: [tid:0] Fetch queue entry created (1/32).
3470000: system.cpu.fetch: [tid:0] Done fetching, reached fetch bandwidth for this cycle.
3470000: system.cpu.fetch: [tid:0] [sn:3123] Sending instruction to decode from fetch queue. Fetch queue size: 1.
3470000: system.cpu.decode: Processing [tid:0]
3470000: system.cpu.decode: [tid:0] Not blocked, so attempting to run stage.
3470000: system.cpu.decode: [tid:0] Sending instruction to rename.
3470000: system.cpu.decode: [tid:0] Processing instruction [sn:3122] with PC (0x100525c=>0x1005262).(2=>3)
3470000: system.cpu.rename: Processing [tid:0]
3470000: system.cpu.rename: [tid:0] Free IQ: 16, Free ROB: 28, Free LQ: 8, Free SQ: 8, FreeRM 31(213 208 255 510 31 2
3470000: system.cpu.rename: [tid:0] 3 instructions not yet in ROB
3470000: system.cpu.rename: calcFreeLQEntries: free lqEntries: 8, loadsInProgress: 0, loads dispatchedToLQ: 0
3470000: system.cpu.rename: [tid:0] Not blocked, so attempting to run stage.
3470000: system.cpu.rename: [tid:0] 1 available instructions to send iew.
3470000: system.cpu.rename: [tid:0] 3 insts pipelining from Rename | 1 insts dispatched to IQ last cycle.
3470000: system.cpu.rename: [tid:0] Sending instructions to IEW.
3470000: system.cpu.rename: [tid:0] Processing instruction [sn:3121] with PC (0x100525c=>0x1005262).(1=>2).
3470000: global: Renamed reg t2 to physical reg 65 (65) old mapping was 43 (43)
3470000: system.cpu.rename: [tid:0] Renaming arch reg 18 (integer) to physical reg 65 (65).
3470000: system.cpu.rename: [tid:0] [sn:3121] Adding instruction to history buffer (size=11).
3470000: system.cpu.rename: [tid:0] Removing a committed instruction from the history buffer 0 (size=11), until [sn:31
3470000: system.cpu.rename: [tid:0] Freeing up older rename of reg 62 (integer), [sn:3115].
3470000: system.cpu.iew: Issue: Processing [tid:0]
3470000: system.cpu.iew: [tid:0] Not blocked, so attempting to run dispatch.
3470000: system.cpu.iew: [tid:0] Issue: Adding PC (0x1005259=>0x100525c).(1=>2) [sn:3119] [tid:0] to IQ.
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Note. Each line is one stage's action at one tick; the 500-tick spacing between groups is one clock cycle.

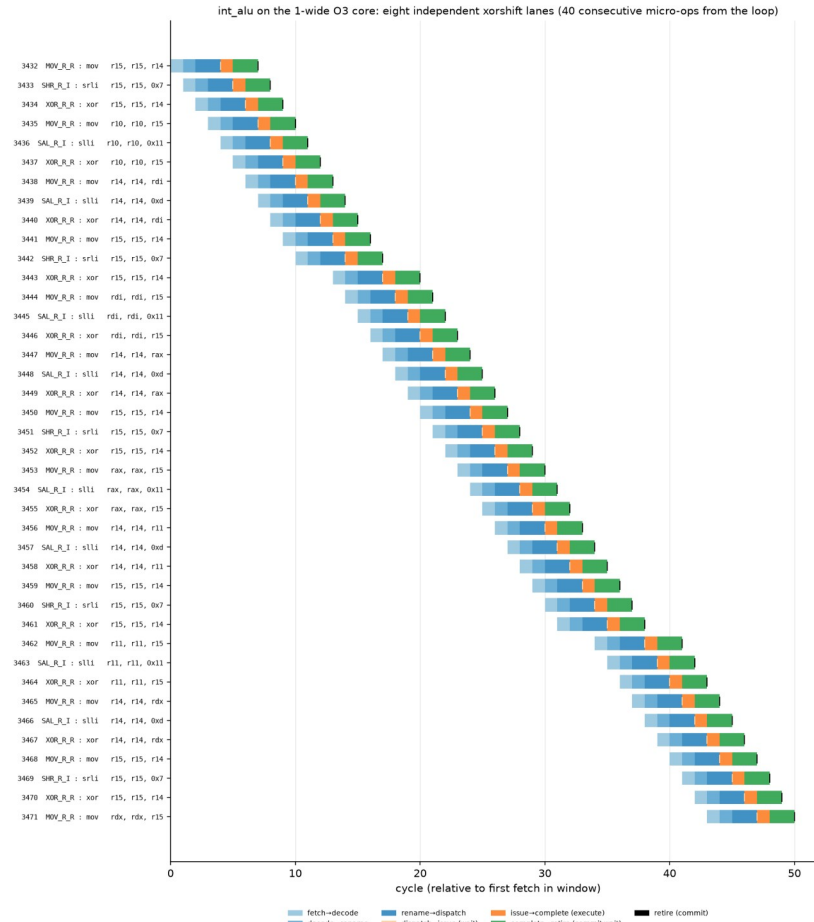
Figure 15

Pipeline Occupancy of 36 Consecutive μ ops of the dep_chain Loop on the 1-Wide Core



Note. Light-to-dark blue segments are the fetch→decode, decode→rename and rename→dispatch delays (fixed at 1, 1 and 2 cycles); pale orange is the wait in the instruction queue; orange is execution; green is the wait for in-order commit. The staircase shape shows the single-issue front end handing one μ op per cycle to the back end; the longer orange segments of the mul1s μ op (3-cycle multiply) and the pale-orange waits of the μ ops behind it show the dependency chain.

Figure 16
Pipeline Occupancy of 40 Consecutive μ ops of the `int_alu` Loop on the 1-Wide Core



Note. With eight independent lanes almost no μ op waits for an operand: the pale-orange (dispatch→issue) segments vanish and every μ op follows the fixed seven-cycle path, so the 1-wide core is limited purely by its fetch/decode bandwidth.

Table 3
The Fully Traced Demonstration Runs (Whole Program, Including C-Library Start-Up)

Workload	Core	Cycles	Instructions	IPC	μ ops fetched	μ ops committed	Squashed	Mean latency (cycles)
dep_chain (30 iterations)	1-wide	21,793	4,032	0.185	9,256	7,249	21.7%	24.6
int_alu (30 iterations)	1-wide	24,339	5,976	0.245	11,098	9,088	18.1%	19.5
dep_chain (30 iterations)	4-wide	15,251	4,032	0.264	13,760	7,250	47.3%	34.3
int_alu (30 iterations)	4-wide	17,089	5,976	0.350	15,917	9,089	42.9%	36.1

Note. These runs are dominated by libc start-up (about 4,000 instructions) and are used only to illustrate the stage flow; the steady-state measurements are in Sections 3 to 5.

Two things stand out in the cycle-by-cycle view. First, the fixed part of the path is seven cycles: one cycle each for fetch→decode, decode→rename, two for rename→dispatch, one for execution and at least two from completion to retirement, so no μop can commit sooner than seven cycles after it was fetched. Second, the two workloads wait in different places. In `dep_chain` every multiply must wait for the previous iteration’s result, so μops queue in the instruction queue (pale orange) and the retire markers spread out; in `int_alu` the eight lanes are independent, nothing waits, and the only limit is that the 1-wide front end can supply one μop per cycle. That contrast is the whole story of ILP: the machine can only go as fast as the program’s dependences and its own narrowest stage allow.

Performance Metrics: Throughput and Latency

For steady-state numbers each of the 28 issue-width runs was repeated with an O3PipeView trace window covering about 40,000 μops from the middle of the run, and `pipestats.py` discarded the first and last 10% of the window (Figure 17). Table 4 lists throughput and latency for the basic 1-wide pipeline and, for comparison, the 4-wide core; Figure 18 shows the full table for all four widths, Figure 19 plots the latency trend and Figure 20 breaks the latency into stage-to-stage delays. Figure 21 shows the raw counters behind the IPC and branch numbers of one run.

Table 4

Throughput and Per- μop Fetch-to-Commit Latency (Steady-State Window) on the 1-Wide and 4-Wide Cores

Workload	IPC 1-wide	Mean lat.	Median	p90	Squashed	IPC 4-wide	Mean lat.	Squashed
<code>int_alu</code>	0.867	7.0	7	7	0.0%	2.712	51.5	0.0%
<code>dep_chain</code>	0.499	7.9	7	9	0.0%	0.887	96.2	0.0%
<code>fp_matmul</code>	0.519	21.5	22	22	2.5%	1.458	43.0	7.6%
<code>mem_stream</code>	0.220	147.4	147	147	0.0%	0.376	63.8	3.3%
<code>mem_chase</code>	0.270	249.3	309	320	0.0%	0.312	665.3	0.0%

Workload	IPC 1-wide	Mean lat.	Median	p90	Squashed	IPC 4-wide	Mean lat.	Squashed
branchy	0.375	9.4	9	12	10.7%	0.769	11.6	36.5%
bsort	0.512	15.9	16	17	8.2%	1.377	15.3	31.4%

Note. Latency in cycles per μop ; IPC in instructions per cycle over the whole run. “Squashed” is the share of fetched μops that never committed (wrong-path work).

Figure 17

pipestats.py Output for the 4-Wide int_alu Trace Window: Throughput, Latency Distribution and Stage Delays

```

gem5-ilp-assignment4 — zsh — 118x18

srincherukuru@Sriman-macbook gem5-ilp-assignment4 % /private/tmp/claude-501/Users-srincherukuru-Documents-Bectran
Workspace/93be3428-3876-46bd-9c28-435fc16ac7db/scratchpad/venv/bin/python scripts/pipestats.py results/runs/pv_w4_int_
alu/trace.out --trim 0.1
instructions_fetched      33656
instructions_committed    33656
squashed_fraction        0.0
window_instructions      33656
window_cycles            11853.0
throughput_ipc           2.8394
latency_mean_cycles      51.526
latency_median_cycles    51.0
latency_p90_cycles       55.0
latency_max_cycles       58.0
stage_delays_cycles      {'fetch->decode': 9.635, 'decode->rename': 1.0, 'rename->dispatch': 2.182, 'dispatch->iss
ue': 17.189, 'issue->complete': 1.0, 'complete->retire': 20.52}
srincherukuru@Sriman-macbook gem5-ilp-assignment4 %

```


Figure 18

The Complete Latency Table for All Seven Workloads at Widths 1, 2, 4 and 8 (latency_table.py)

```

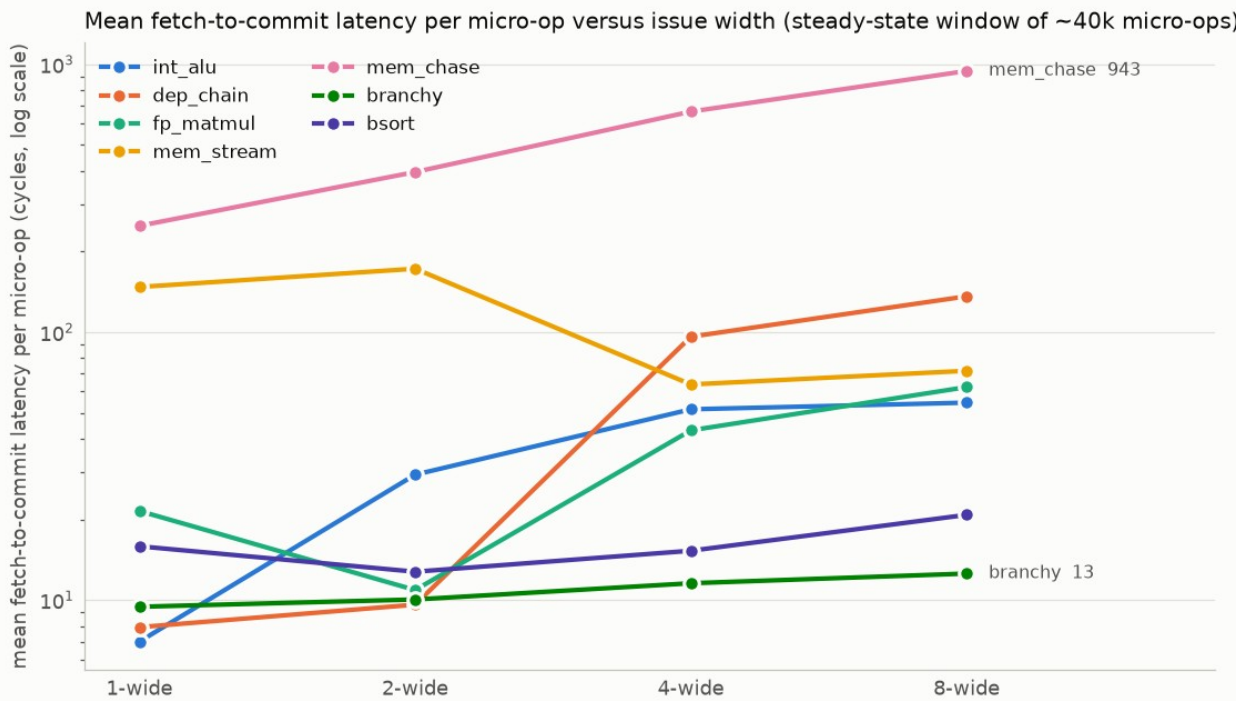
gem5-ilp-assignment4 --zsh -- 150x36

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % /private/tmp/claude-501/-Users-srimancherukuru-Documents-BectranWorkspace/93be3428-3876-46bd-9c2
8-435fc16ac7db/scratchpad/venv/bin/python scripts/latency_table.py | cut -c1-148
wrote /Users/srimancherukuru/git/gem5-ilp-assignment4/results/latency.csv (28 rows)
workload  w uPC(win) IPC(win) IPC(run) lat mean median p90 squash% f>d d>n n>p p>i i>c c>r
branchy   1 0.765 0.376 0.376 9.4 9.0 12 10.7% 1.4 1.0 2.0 0.8 1.0 3.3
branchy   2 1.191 0.585 0.585 10.1 10.0 12 19.8% 1.7 1.0 2.0 1.1 1.0 3.3
branchy   4 1.563 0.768 0.769 11.6 11.0 15 36.5% 1.9 1.0 2.0 1.2 1.0 4.4
branchy   8 1.918 0.943 0.917 12.6 12.0 16 46.3% 1.8 1.0 2.1 1.5 1.0 5.2
bsort     1 0.912 0.521 0.512 15.9 16.0 17 8.2% 5.5 1.0 2.0 1.6 1.0 4.7
bsort     2 1.550 0.885 0.894 12.8 13.0 15 16.2% 3.3 1.0 2.1 1.7 1.0 3.7
bsort     4 2.357 1.346 1.377 15.3 15.0 18 31.4% 4.7 1.0 2.2 1.6 1.0 4.9
bsort     8 2.504 1.430 1.486 20.8 19.0 29 51.5% 4.8 1.0 2.5 3.4 1.0 8.1
dep_chain 1 0.812 0.500 0.499 7.9 7.0 9 0.0% 1.0 1.0 2.0 0.8 1.2 2.0
dep_chain 2 1.300 0.800 0.798 9.6 10.0 10 0.0% 1.0 1.0 2.0 2.0 1.2 2.5
dep_chain 4 1.441 0.886 0.887 96.2 96.0 98 0.0% 23.3 1.0 2.9 40.8 1.2 27.0
dep_chain 8 1.439 0.885 0.887 136.0 136.0 140 0.0% 25.2 1.0 4.7 63.2 1.2 40.8
fp_matmul 1 0.969 0.525 0.519 21.5 22.0 22 2.5% 5.9 1.0 2.0 2.9 1.1 8.6
fp_matmul 2 1.787 0.969 0.945 10.9 10.0 13 2.7% 1.2 1.0 2.0 1.8 1.1 3.8
fp_matmul 4 2.825 1.531 1.458 43.0 41.0 60 7.6% 4.0 1.0 2.1 7.2 1.1 27.7
fp_matmul 8 3.325 1.802 1.690 62.4 59.0 89 7.6% 7.9 1.0 2.5 9.7 1.1 40.2
int_alu   1 0.906 0.870 0.867 7.0 7.0 7 0.0% 1.0 1.0 2.0 0.0 1.0 2.0
int_alu   2 1.671 1.605 1.598 29.4 29.0 32 0.0% 1.0 1.0 2.0 12.4 1.0 11.9
int_alu   4 2.839 2.727 2.712 51.5 51.0 55 0.0% 9.6 1.0 2.2 17.2 1.0 20.5
int_alu   8 3.873 3.720 3.695 54.6 54.0 59 0.0% 7.8 1.0 2.4 19.6 1.0 22.7
mem_chase 1 0.263 0.145 0.270 249.3 309.0 320 0.0% 126.7 1.0 4.8 27.9 1.0 87.8
mem_chase 2 0.250 0.138 0.311 395.8 397.0 412 0.0% 139.0 1.0 6.2 68.6 1.0 180.0
mem_chase 4 0.250 0.138 0.312 665.3 696.0 724 0.0% 151.5 1.0 8.9 141.0 1.0 361.9
mem_chase 8 0.248 0.137 0.312 943.1 920.0 1051 0.0% 161.8 1.0 18.5 214.0 1.0 546.8
mem_stream 1 0.439 0.212 0.220 147.4 147.0 147 0.0% 75.3 1.0 3.3 6.4 1.0 60.4
mem_stream 2 0.565 0.272 0.259 172.5 132.0 216 0.0% 59.6 1.0 3.3 11.0 1.0 96.5
mem_stream 4 0.491 0.237 0.376 63.8 90.0 102 3.3% 51.1 1.0 7.8 0.3 1.0 2.5
mem_stream 8 0.491 0.236 0.445 71.7 102.0 112 4.3% 57.9 1.0 8.5 0.5 1.0 2.8
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Note. uPC is the trace-derived μop throughput; IPC(win) divides it by the μop ratio and agrees with the whole-run IPC(run) from stats.txt.

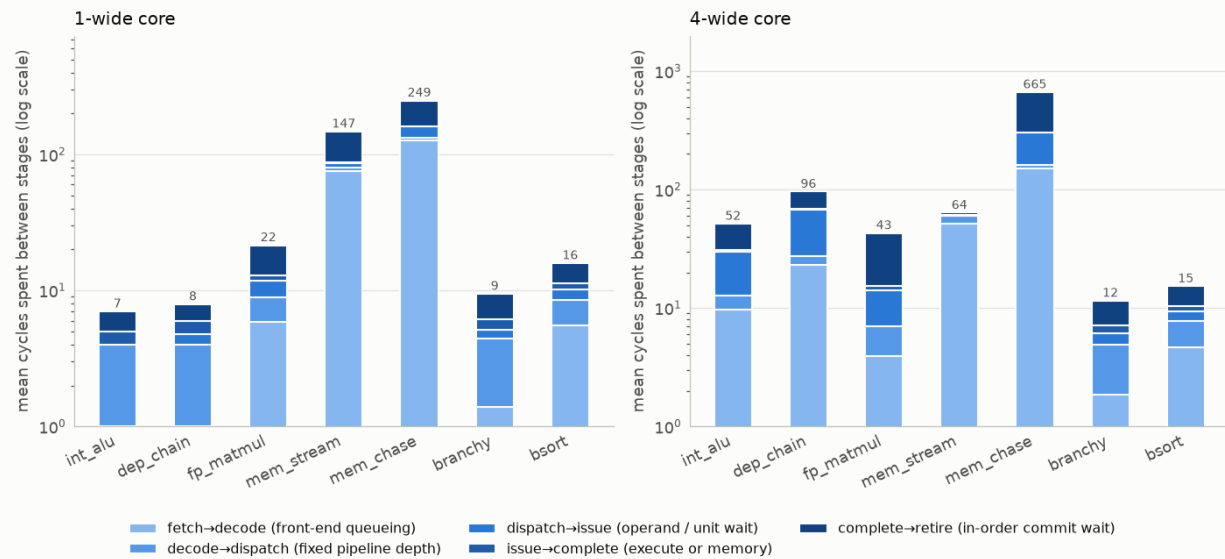
Figure 19
Mean Fetch-to-Commit Latency per μ op Versus Issue Width (Log Scale)



Note. Latency rises with width for every workload whose throughput does not rise as fast, because the larger window holds more waiting μ ops (Little's law).

Figure 20
Where the Latency Goes: Mean Delay Between Pipeline Events on the 1-Wide and 4-Wide Cores

Where an instruction's latency goes: mean time between pipeline events, per workload



Note. For the memory-bound workloads most of the time is spent queued in the front end behind a full window (fetch $\rightarrow$ decode) and waiting to commit behind an older miss (complete $\rightarrow$ retire).

Figure 21

Raw Counters From stats.txt for the 4-Wide bsort Run: Cycles, IPC/CPI, Committed Instructions and μ ops, Predictor and Squash Statistics

```

srincherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -E '^system\.cpu\.('numCycles|cpi|commitStats0\.(numInst
s|numOps)|branchPred\.(condPredicted|condIncorrect|BTBHitRatio|lookups_0::total)|commit\.(branchMispredicts|commitSqua
shedInsts)|numSquashedInsts|rename\.(ROBFullEvents|IQFullEvents|SQFullEvents)|idleCycles)\b' results/runs/ss_w4_bsort/
stats.txt | cut -c1-116
system.cpu.numCycles                1875401                # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                      0.726439              # CPI: cycles per instruction (core leve
system.cpu.ipc                      1.376578              # IPC: instructions per cycle (core leve
system.cpu.numSquashedInsts         54868                 # Number of squashed instructions skippe
system.cpu.idleCycles               7837                  # Total number of cycles that the CPU ha
system.cpu.branchPred.lookups_0::total 1123160              # Number of BP lookups (Count)
system.cpu.branchPred.condPredicted 888567                # Number of conditional branches predict
system.cpu.branchPred.condIncorrect 35597                 # Number of conditional branches incorre
system.cpu.branchPred.BTBHitRatio    0.996787              # BTB Hit Ratio (Ratio)
system.cpu.commit.commitSquashedInsts 950613                # The number of squashed insts skipped b
system.cpu.commit.branchMispredicts 35631                 # The number of times a branch was mispr
system.cpu.commitStats0.numInsts     2581636               # Number of instructions committed (thre
system.cpu.commitStats0.numOps       4520690               # Number of ops (including micro ops) co
system.cpu.rename.ROBFullEvents      27                    # Number of times rename has blocked due
system.cpu.rename.IQFullEvents       37                    # Number of times rename has blocked due
system.cpu.rename.SQFullEvents       4680                  # Number of times rename has blocked due
srincherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

The patterns are instructive. On the 1-wide core the ALU-bound `int_alu` achieves 0.87 IPC with the minimum seven-cycle latency, `dep_chain` reaches only 0.50 IPC although its latency is almost as low (its μ ops wait in the queue, not in the pipeline), and the memory-bound programs have both low IPC (0.22 and 0.27) and long latencies (147 and 249 cycles) because every μ op behind a DRAM miss waits for it. Widening the core exposes the throughput–latency trade-off: `dep_chain`’s IPC rises only from 0.50 to 0.89, yet its mean latency jumps from 7.9 cycles at 1-wide to 96 cycles at 4-wide and 136 at 8-wide. The reason is Little’s law: a 128-entry ROB divided by 1.44 μ ops per cycle is about 89 cycles of residence per μ op, which matches the measurement. The front end runs far ahead, fills the window with μ ops that cannot execute, and each of them waits its turn. For `mem_chase` the window fills with μ ops that all depend on one pointer load, so latency grows from 249 to 943 cycles while IPC stays at 0.31. The squashed fraction grows with width for the branchy programs (bsort: 8% at 1-wide, 52% at 8-wide)

because a wider machine fetches more wrong-path work before a misprediction is discovered, which foreshadows the next section.

Impact of Branch Prediction

Five direction predictors were compared on the 4-wide core: the two static predictors (always not-taken, always taken), a 2-bit local predictor with 2,048 counters (gem5’s LocalBP), the default tournament predictor (2,048-entry local, 8,192-entry global and choice tables) and TAGE. The BTB, return-address stack and indirect predictor were identical in all runs, so the static configurations are the pipeline “without branch prediction” in the sense of having no learned direction prediction: unconditional branches still have their targets buffered. Three workloads were used, the two branch-heavy programs and, as a control, `int_alu` whose only branch is a loop back-edge. The static not-taken and tournament predictors were also run on the 1-wide core to see how the value of prediction depends on the machine width.

Table 5
Branch Predictors on the 4-Wide Core

Workload	Predictor	IPC	CPI	Cond. mispredicted	MPKI	Wasted fetch
branchy	static not-taken	0.723	1.383	17.4%	46.1	73.1%
branchy	static taken	0.456	2.195	32.4%	122.6	107.5%
branchy	2-bit local	0.745	1.343	16.3%	42.1	59.3%
branchy	tournament	0.769	1.300	15.8%	39.9	55.9%
branchy	TAGE	0.783	1.277	15.3%	38.2	53.3%
bsort	static not-taken	0.704	1.420	9.8%	62.1	173.0%
bsort	static taken	0.378	2.647	22.8%	186.1	293.0%
bsort	2-bit local	0.939	1.065	7.1%	35.0	107.4%
bsort	tournament	1.377	0.726	4.0%	13.8	41.4%
bsort	TAGE	1.741	0.574	1.6%	4.5	12.5%
int_alu	static not-taken	2.057	0.486	97.8%	13.6	24.0%
int_alu	static taken	2.711	0.369	0.4%	0.1	0.1%
int_alu	2-bit local	2.713	0.369	0.4%	0.0	0.1%
int_alu	tournament	2.712	0.369	0.4%	0.1	0.1%
int_alu	TAGE	2.712	0.369	0.3%	0.0	0.1%

Note. MPKI = branch mispredictions per thousand committed instructions. “Wasted fetch” = instructions fetched beyond those committed, i.e., wrong-path fetch as a share of useful work.

Figure 22

The Branch-Predictor Runs as Aggregated by parse_stats.py (Including the 1-Wide Runs)

```

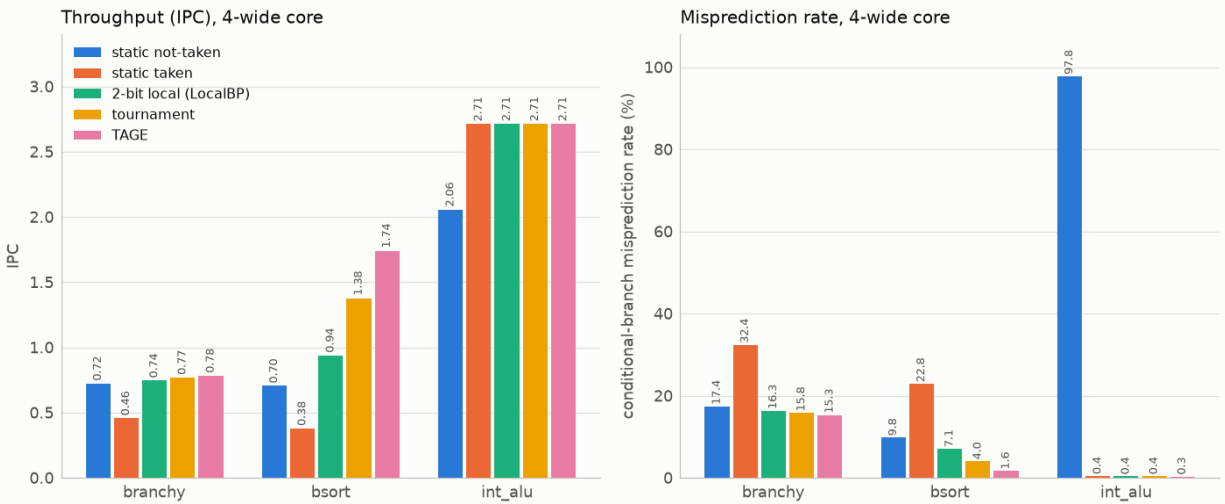
gem5-ilp-assignment4 — -zsh — 132x30

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % python3 scripts/parse_stats.py | grep -E '^ (run|bp_)' | cut -c1-130
run
thr      cycles      insts      IPC      CPI      mispr%      MPKI      output
bp_local_w4_branchy      1      5,227,045      3,892,612      0.745      1.343      16.32%      42.09      branchy: passes=20 checksum=
bp_local_w4_bsort      1      2,750,772      2,581,636      0.939      1.065      7.06%      35.05      bsort: n=800 sorted=1 checks
bp_local_w4_int_alu      1      1,365,434      3,703,879      2.713      0.369      0.36%      0.04      int_alu: n=50000 checksum=da
bp_static-nt_w1_branchy      1      10,604,752      3,892,612      0.367      2.724      26.05%      46.14      branchy: passes=20 checksum=
bp_static-nt_w1_bsort      1      6,618,148      2,581,636      0.390      2.563      19.98%      62.07      bsort: n=800 sorted=1 checks
bp_static-nt_w1_int_alu      1      4,673,402      3,703,879      0.792      1.262      99.11%      13.58      int_alu: n=50000 checksum=da
bp_static-nt_w4_branchy      1      5,383,123      3,892,612      0.723      1.383      17.38%      46.14      branchy: passes=20 checksum=
bp_static-nt_w4_bsort      1      3,666,078      2,581,636      0.704      1.420      9.79%      62.08      bsort: n=800 sorted=1 checks
bp_static-nt_w4_int_alu      1      1,800,762      3,703,879      2.057      0.486      97.79%      13.59      int_alu: n=50000 checksum=da
bp_static-t_w4_branchy      1      8,543,883      3,892,612      0.456      2.195      32.38%      122.63      branchy: passes=20 checksum=
bp_static-t_w4_bsort      1      6,833,793      2,581,636      0.378      2.647      22.83%      186.06      bsort: n=800 sorted=1 checks
bp_static-t_w4_int_alu      1      1,366,212      3,703,879      2.711      0.369      0.44%      0.05      int_alu: n=50000 checksum=da
bp_tag_w4_branchy      1      4,971,075      3,892,612      0.783      1.277      15.28%      38.20      branchy: passes=20 checksum=
bp_tag_w4_bsort      1      1,482,931      2,581,636      1.741      0.574      1.62%      4.48      bsort: n=800 sorted=1 checks
bp_tag_w4_int_alu      1      1,365,645      3,703,879      2.712      0.369      0.32%      0.04      int_alu: n=50000 checksum=da
bp_tournament_w1_branchy      1      10,374,131      3,892,612      0.375      2.665      23.08%      39.92      branchy: passes=20 checksum=
bp_tournament_w1_bsort      1      5,037,165      2,581,636      0.512      1.951      5.08%      13.62      bsort: n=800 sorted=1 checks
bp_tournament_w1_int_alu      1      4,271,922      3,703,879      0.867      1.153      0.42%      0.05      int_alu: n=50000 checksum=da
bp_tournament_w4_branchy      1      5,060,608      3,892,612      0.769      1.300      15.76%      39.94      branchy: passes=20 checksum=
bp_tournament_w4_bsort      1      1,875,401      2,581,636      1.377      0.726      4.01%      13.80      bsort: n=800 sorted=1 checks
bp_tournament_w4_int_alu      1      1,365,844      3,703,879      2.712      0.369      0.39%      0.05      int_alu: n=50000 checksum=da
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

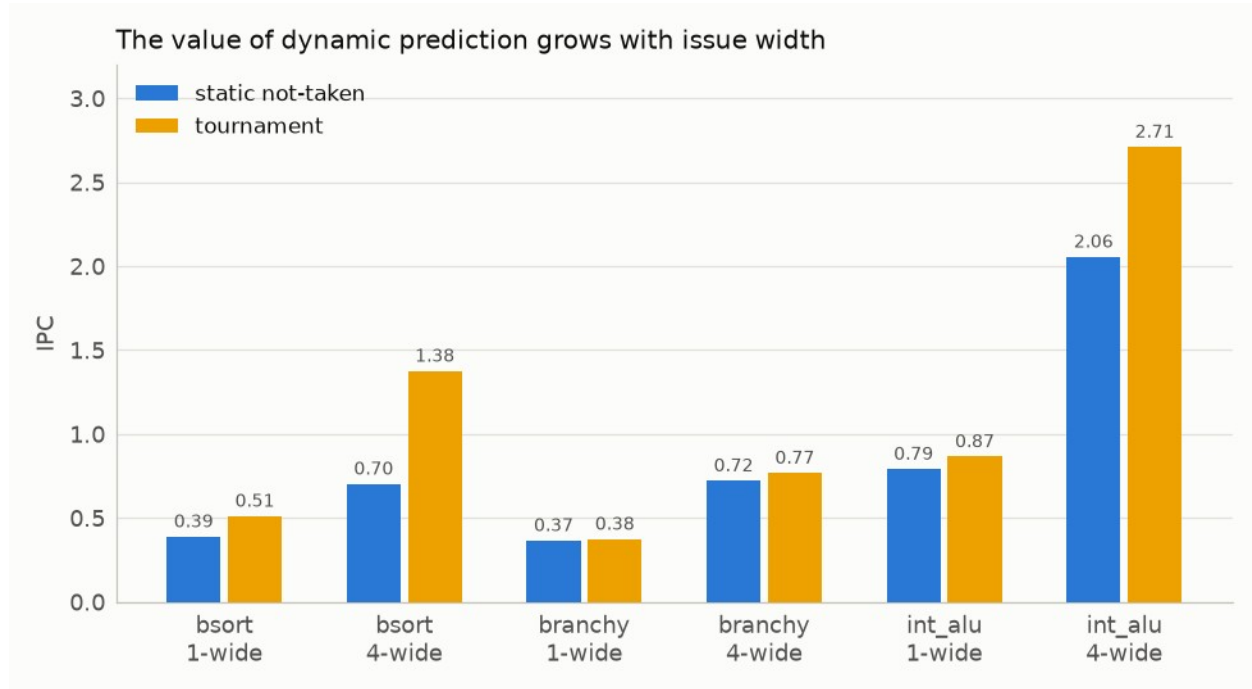
Figure 23

Throughput and Conditional-Branch Misprediction Rate by Predictor and Workload (4-Wide Core)



Note. Static taken is the worst predictor for these loops because most of their conditional branches fall through; the static not-taken bar for int_alu is 97.8% because its only branch is an always-taken loop back-edge.

Figure 24
The Same Two Predictors on the 1-Wide and 4-Wide Cores



Note. The gain from dynamic prediction is small on the narrow core and large on the wide one.

The results separate the workloads by how predictable they are. The branches in *branchy* are decided by random data, so no predictor can do better than the entropy of the data allows: every dynamic predictor sits at 15–16% mispredicted (about 40 MPKI) and gains only 3–8% IPC over static not-taken, while static taken doubles the misprediction rate and cuts IPC by a third. Bubble sort is the opposite case. Its comparison branches look random in the first passes but become highly repetitive as the array gets sorted, so the more history a predictor keeps the better it does: 9.8% mispredicted for static not-taken, 7.1% for the 2-bit local predictor, 4.0% for tournament and 1.6% for TAGE, and IPC climbs from 0.70 to 0.94, 1.38 and 1.74, a 2.5× range from prediction alone. For *int_alu* the loop branch is taken 49,999 times out of 50,000; static not-taken mispredicts every iteration (97.8%), but because the loop body is 50 instructions long the damage is limited to 24% of IPC (2.06 versus 2.71), and every dynamic predictor learns the branch after one iteration.

Why accurate prediction is crucial becomes clear when width changes (Figure 24). On the 1-wide core the tournament predictor lifts bsort by 31% over static not-taken (0.39 to 0.51 IPC); on the 4-wide core the lift is 96% (0.70 to 1.38). The misprediction penalty in cycles, roughly the depth from fetch to branch resolution, is the same on both machines, but the wide machine loses four issue slots per penalty cycle instead of one and it also fetches and renames far more wrong-path work before the branch resolves (Table 5's wasted-fetch column). Branch prediction is therefore not an independent optimization: it is what makes width usable, and a superscalar core without a good predictor spends most of its bandwidth on instructions it will throw away.

Multiple Issue (Superscalar) Simulation

The 1-wide baseline was widened to 2, 4 and 8 with every stage width, the ROB, IQ and load/store queues and the functional-unit pool scaled together (Table 1). All seven workloads were run at all four widths with the tournament predictor; Figures 25 and 26 show two of the runs, Figure 27 the aggregated table, and Figures 28 and 29 the results as charts.

Figure 25

Running int_alu on the 1-Wide Core: the Configuration Banner, the Program's Own Output and the Exit Message

```
gem5-ilp-assignment4 — zsh — 118x18

srmancherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -v '^warn' results/runs/ss_w1_int_alu/run.log | tail -14 | cut -c1-116

gem5 version 25.1.0.1
gem5 compiled Sep 13 2026 13:17:59
gem5 started Sep 13 2026 13:30:31
gem5 executing on Sriman-macbook.local, pid 57444
command line: /Users/srmancherukuru/git/gem5/build/X86/gem5.opt --outdir=/Users/srmancherukuru/git/gem5-ilp-assign

info: Standard input is not a terminal, disabling listeners.
[ilp_config] cpu=o3 width=1 bp=tournament threads=1 cmds=['bin/int_alu:50000']
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.hh:817: warn: Unsupported ioctl call (return ENOTTY): ioctl(1, 0x5413, ...) @ (0x1002892=>0x100
int_alu: n=50000 checksum=da8cf086078afda2
[ilp_config] exiting @ tick 2135960500 because exiting with last active thread context
srmancherukuru@Sriman-macbook gem5-ilp-assignment4 %
```


Figure 26

The Same Program on the 8-Wide Core: Identical Output and Checksum, 4.3 Times Fewer Ticks

```

gem5-ilp-assignment4 — -zsh — 118x18

srmancherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -v '^warn' results/runs/ss_w8_int_alu/run.log | tail -14 | cut -c1-116

gem5 version 25.1.0.1
gem5 compiled Sep 13 2026 13:17:59
gem5 started Sep 13 2026 13:30:31
gem5 executing on Sriman-macbook.local, pid 57425
command line: /Users/srmancherukuru/git/gem5/build/X86/gem5.opt --outdir=/Users/srmancherukuru/git/gem5-ilp-assign

info: Standard input is not a terminal, disabling listeners.
[ilp_config] cpu=o3 width=8 bp=tournament threads=1 cmds=['bin/int_alu:50000']
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.hh:817: warn: Unsupported ioctl call (return ENOTTY): ioctl(1, 0x5413, ...) @ (0x1002892=>0x100
int_alu: n=50000 checksum=da8cf086078afda2
[ilp_config] exiting @ tick 501236500 because exiting with last active thread context
srmancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Figure 27

The 28 Issue-Width Runs Aggregated by parse_stats.py

```

gem5-ilp-assignment4 — -zsh — 132x36

srmancherukuru@Sriman-macbook gem5-ilp-assignment4 % python3 scripts/parse_stats.py | grep -E '^(run|ss_w)' | cut -c1-130
run
ss_w1_branchy      thr 1 10,374,131 3,892,612 0.375 2.665 23.08% 39.92 branchy: passes=20 checksum=
ss_w1_bsort        1 5,037,165 2,581,636 0.512 1.951 5.08% 13.62 bsort: n=800 sorted=1 checks
ss_w1_dep_chain    1 8,021,884 4,003,955 0.499 2.003 0.04% 0.04 dep_chain: n=500000 checksum
ss_w1_fp_matmul    1 6,295,875 3,268,564 0.519 1.926 2.08% 2.97 fp_matmul: reps=4 checksum=2
ss_w1_int_alu      1 4,271,922 3,703,879 0.867 1.153 0.42% 0.05 int_alu: n=50000 checksum=da
ss_w1_mem_chase    1 16,142,730 4,349,775 0.270 3.711 0.03% 0.04 mem_chase: hops=300000 final
ss_w1_mem_stream   1 31,054,826 6,819,943 0.220 4.553 0.02% 0.03 mem_stream: passes=2 checksu
ss_w2_branchy      1 6,656,082 3,892,612 0.585 1.710 22.43% 39.92 branchy: passes=20 checksum=
ss_w2_bsort        1 2,888,830 2,581,636 0.894 1.119 4.79% 13.65 bsort: n=800 sorted=1 checks
ss_w2_dep_chain    1 5,017,991 4,003,955 0.798 1.253 0.04% 0.04 dep_chain: n=500000 checksum
ss_w2_fp_matmul    1 3,458,890 3,268,564 0.945 1.058 2.08% 2.97 fp_matmul: reps=4 checksum=2
ss_w2_int_alu      1 2,317,855 3,703,879 1.598 0.626 0.40% 0.05 int_alu: n=50000 checksum=da
ss_w2_mem_chase    1 13,973,083 4,349,775 0.311 3.212 0.03% 0.04 mem_chase: hops=300000 final
ss_w2_mem_stream   1 26,320,324 6,819,943 0.259 3.859 0.01% 0.03 mem_stream: passes=2 checksu
ss_w4_branchy      1 5,060,608 3,892,612 0.769 1.300 15.76% 39.94 branchy: passes=20 checksum=
ss_w4_bsort        1 1,875,401 2,581,636 1.377 0.726 4.01% 13.80 bsort: n=800 sorted=1 checks
ss_w4_dep_chain    1 4,515,074 4,003,955 0.887 1.128 0.04% 0.04 dep_chain: n=500000 checksum
ss_w4_fp_matmul    1 2,242,507 3,268,564 1.458 0.686 1.96% 2.97 fp_matmul: reps=4 checksum=2
ss_w4_int_alu      1 1,365,844 3,703,879 2.712 0.369 0.39% 0.05 int_alu: n=50000 checksum=da
ss_w4_mem_chase    1 13,959,295 4,349,775 0.312 3.209 0.03% 0.04 mem_chase: hops=300000 final
ss_w4_mem_stream   1 18,161,581 6,819,943 0.376 2.663 0.01% 0.03 mem_stream: passes=2 checksu
ss_w8_branchy      1 4,246,971 3,892,612 0.917 1.091 13.78% 39.92 branchy: passes=20 checksum=
ss_w8_bsort        1 1,737,042 2,581,636 1.486 0.673 2.84% 13.96 bsort: n=800 sorted=1 checks
ss_w8_dep_chain    1 4,514,071 4,003,955 0.887 1.127 0.04% 0.05 dep_chain: n=500000 checksum
ss_w8_fp_matmul    1 1,934,236 3,268,564 1.690 0.592 1.95% 2.97 fp_matmul: reps=4 checksum=2
ss_w8_int_alu      1 1,002,474 3,703,879 3.695 0.271 0.38% 0.05 int_alu: n=50000 checksum=da
ss_w8_mem_chase    1 13,955,094 4,349,775 0.312 3.208 0.03% 0.04 mem_chase: hops=300000 final
ss_w8_mem_stream   1 15,336,955 6,819,943 0.445 2.249 0.01% 0.03 mem_stream: passes=2 checksu
srmancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Table 6

IPC Versus Issue Width and the 8-Wide Speedup Over the 1-Wide Core

Workload	1-wide	2-wide	4-wide	8-wide	Speedup 8/1	CPI 8-wide
int_alu	0.867	1.598	2.712	3.695	4.26×	0.27
dep_chain	0.499	0.798	0.887	0.887	1.78×	1.13
fp_matmul	0.519	0.945	1.458	1.690	3.25×	0.59

Workload	1-wide	2-wide	4-wide	8-wide	Speedup 8/1	CPI 8-wide
mem_stream	0.220	0.259	0.376	0.445	2.03×	2.25
mem_chase	0.270	0.311	0.312	0.312	1.16×	3.21
branchy	0.375	0.585	0.769	0.917	2.44×	1.09
bsort	0.512	0.894	1.377	1.486	2.90×	0.67

Figure 28

IPC of Every Workload at Issue Widths 1, 2, 4 and 8

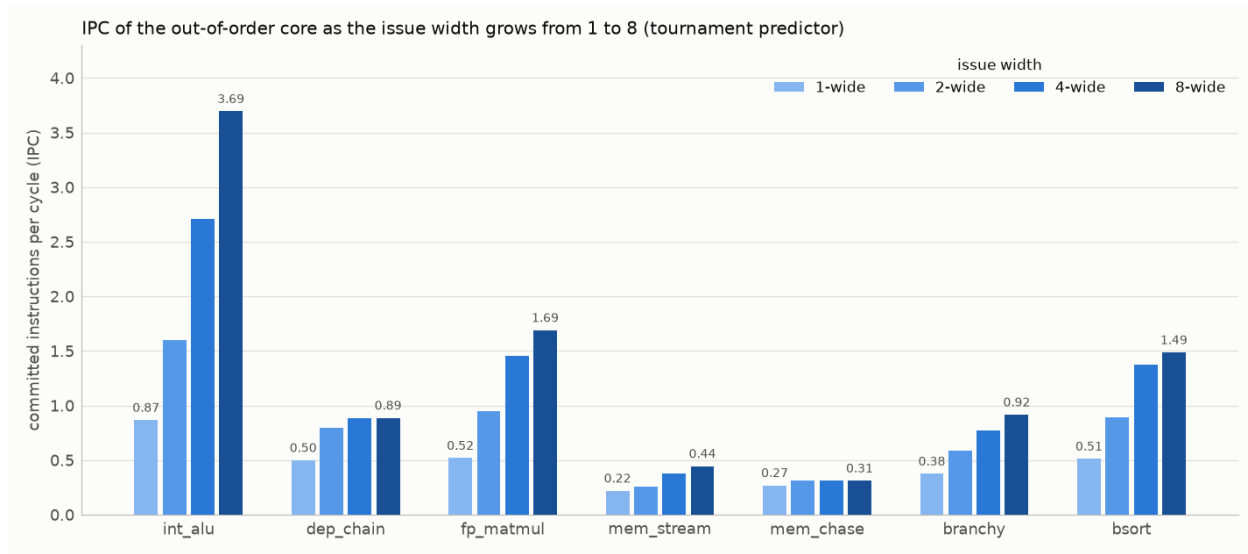
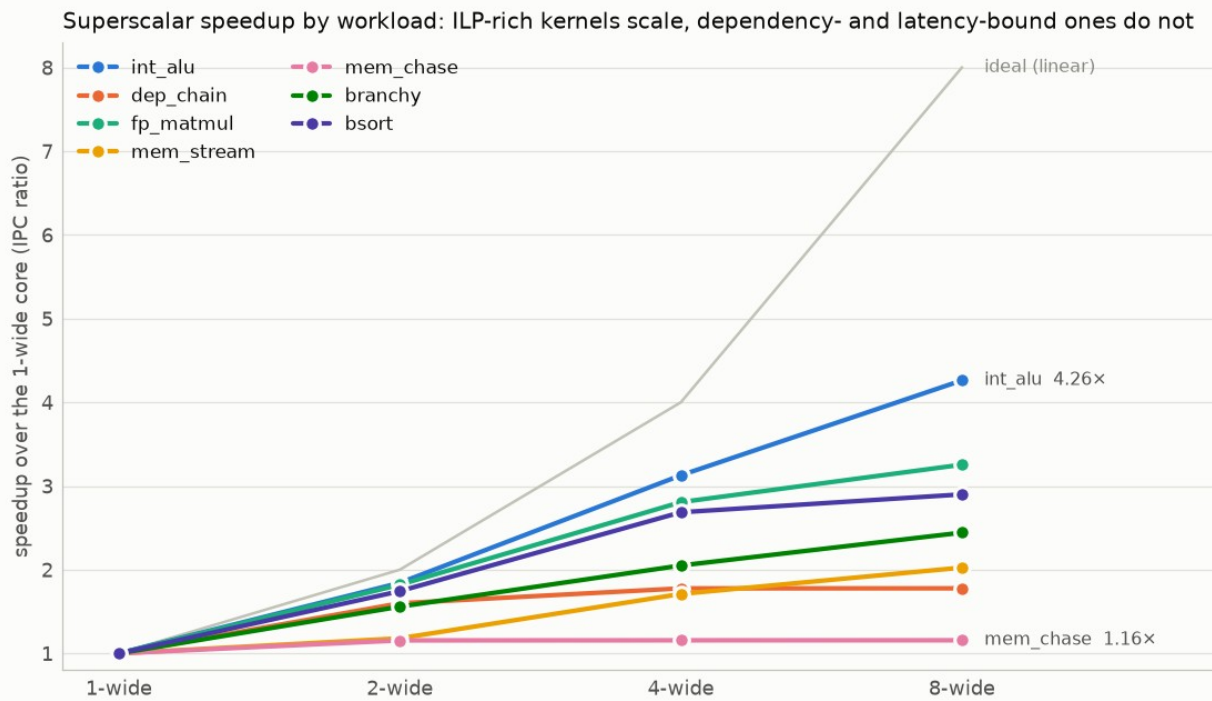


Figure 29
Speedup Over the 1-Wide Core; the Gray Line Is Ideal Linear Scaling



Note. No workload reaches the ideal line; the ranking follows the amount of independent work each program exposes.

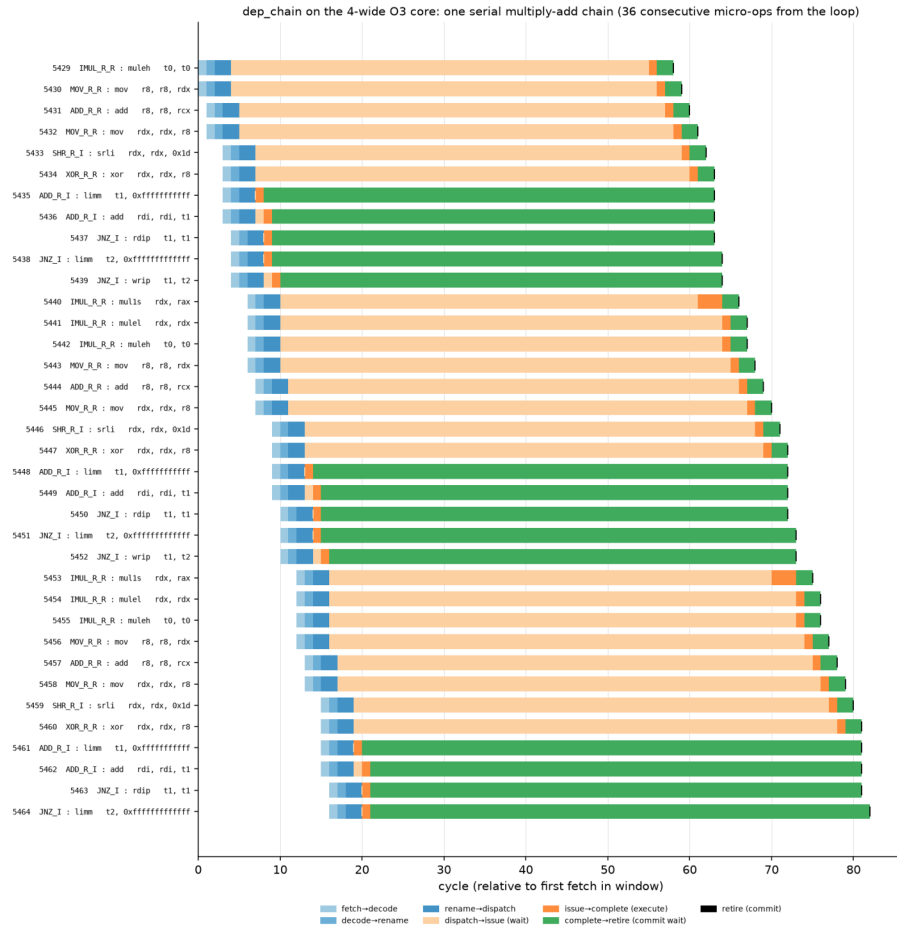
Table 7
Structural Stall Events per Thousand Instructions on the 4-Wide Core (Rename Stage Blocked Because a Buffer Was Full)

Workload	ROB full	IQ full	Store queue full	Load queue full	L1D miss rate
int_alu	40.50	11.83	0.18	0.00	5.46%
dep_chain	0.01	405.84	0.19	0.00	5.47%
fp_matmul	42.24	0.01	21.52	0.00	0.07%
mem_stream	19.46	0.01	1014.19	0.00	70.21%
mem_chase	86.32	0.01	811.08	0.00	40.29%
branchy	0.02	0.28	25.15	0.00	0.51%
bsort	0.01	0.01	1.81	0.00	0.02%

Note. Each workload hits a different wall: the ROB for int_alu and fp_matmul, the instruction queue for the dependency chain (μ ops waiting for operands), the store queue for the memory streams.

Figure 30

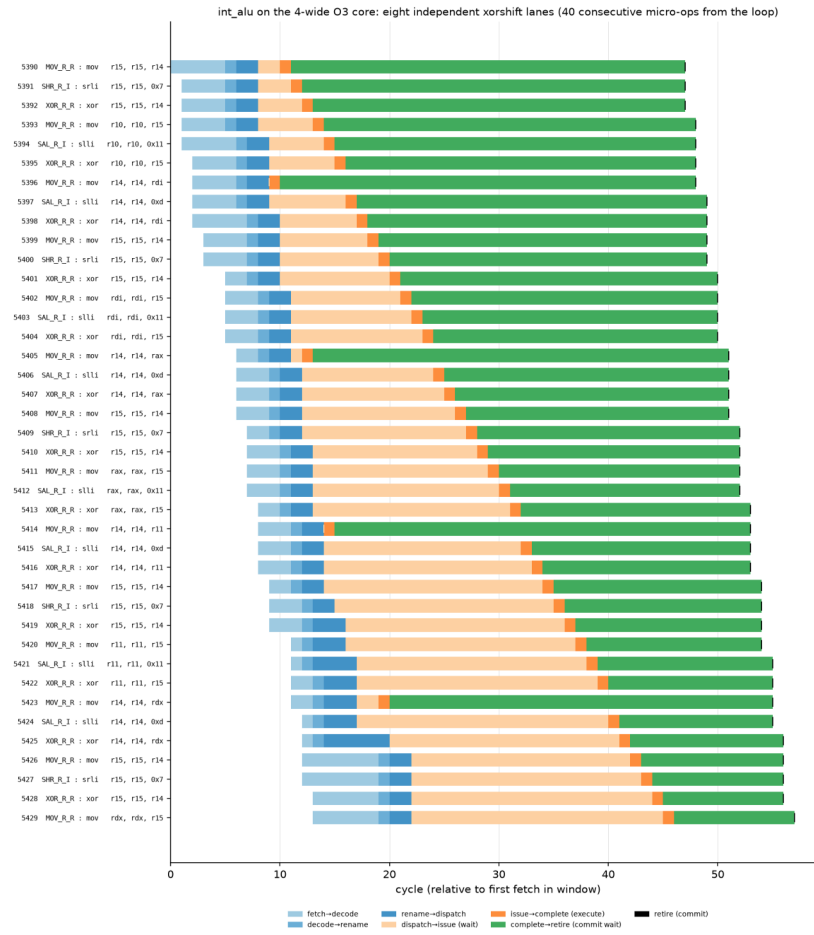
dep_chain on the 4-Wide Core: the Front End Delivers Four μ ops per Cycle but They Queue Behind the Multiply Chain



Note. Compared with Figure 15, fetch, decode and rename now happen in bursts of four, yet the issue (orange) markers keep the same one-multiply-per-iteration spacing and the pale-orange waits grow.

Figure 31

int_alu on the 4-Wide Core: Four μ ops per Cycle Move Through Every Stage Almost Without Waiting



Note. The eight independent lanes give the scheduler enough ready μ ops to fill most of the four issue slots each cycle.

add, a shift and an exclusive-or, and no width can shorten that chain; the instruction queue simply fills with μ ops waiting for it (406 IQ-full events per thousand instructions). `mem_chase` is the extreme case, $1.16\times$ at 8-wide, because every hop is a DRAM access that depends on the previous hop; the wider core only waits with more entries occupied. `mem_stream` doubles ($2.0\times$) not because of width itself but because the store and load queues that come with it let more misses overlap, which is memory-level rather than instruction-level parallelism.

Multithreading (SMT)

For SMT the 4-wide core was given two (or four) hardware threads, one per program, with `gem5`'s default policies: round-robin fetch and commit and partitioned ROB, IQ and load/store queues, so each of two threads owns half of the 128-entry ROB, the 64-entry IQ and the 32-entry load and store queues while the functional units, caches and predictor are shared. Each run stops when the first thread reaches three million committed instructions, which avoids a tail in which one thread runs alone. Throughput is compared with the fairest single-core alternative, running the same instructions one program after the other at each program's single-thread IPC ("sequential IPC"); the ratio of the two is the throughput speedup. Figure 33 shows one run, Figure 34 the per-run metrics and Figure 35 the comparison.

Figure 33

An SMT Run of fp_matmul and int_alu: Both Programs' Outputs and the Per-Thread Committed-Instruction Counters

```

gem5-ilp-assignment4 — -zsh — 118x16

gem5 version 25.1.0.1
gem5 compiled Sep 13 2026 13:17:59
gem5 started Sep 13 2026 13:31:11
gem5 executing on Sriman-macbook.local, pid 57907
command line: /Users/srimancherukuru/git/gem5/build/X86/gem5.opt --outdir=/Users/srimancherukuru/git/gem5-ilp-assign

info: Standard input is not a terminal, disabling listeners.
[ilp_config] cpu=o3 width=4 bp=tournament threads=2 cmds=['bin/fp_matmul:4', 'bin/int_alu:50000']
[ilp_config] exiting @ tick 853163500 because a thread reached the max instruction count
system.cpu.numCycles          1706328          # Number of cpu cycles simulated (Cycle)
system.cpu.commitStats0.numInsts 1416636          # Number of instructions committed (thread level) (Count)
system.cpu.commitStats1.numInsts 3000000          # Number of instructions committed (thread level) (Count)
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Figure 34

smt_table.py: Total and Per-Thread IPC, Sequential Baseline, Throughput and Weighted Speedup, and Buffer-Full Events for Every SMT Run

```

gem5-ilp-assignment4 — -zsh — 132x26

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % python3 scripts/smt_table.py | cut -c1-130
wrote /Users/srimancherukuru/git/gem5-ilp-assignment4/results/smt_summary.csv
run
IPC      seq      x      wsp      t0      t1      t2      t3      ROBfull/k  SQfull/k
smt2_bsort+branchy      1.246  0.966  1.29  1.29  0.576  0.67      3.02  6.89
smt2_bsort+bsort        1.377  1.377  1.00  1.00  0.689  0.689      0.01  0.45
smt2_dep_chain+dep_chain 1.142  0.887  1.29  1.29  0.571  0.571      0.0  0.21
smt2_dep_chain+mem_chase 0.615  0.886  0.69  0.69  0.615  0.0      507.05  0.47
smt2_fp_matmul+int_alu   2.588  2.125  1.22  1.22  0.83  1.758      13.92  13.65
smt2_fp_matmul+mem_chase 1.427  0.578  2.47  2.47  0.837  0.59      5.46  236.32
smt2_int_alu+dep_chain   2.570  1.587  1.62  1.62  1.685  0.885      4.12  0.28
smt2_int_alu+int_alu     2.876  2.712  1.06  1.06  1.438  1.438      0.01  0.2
smt2_int_alu+mem_chase   0.960  2.701  0.36  0.36  0.96  0.001      324.64  0.44
smt2_int_alu+mem_stream  0.961  2.703  0.35  0.35  0.96  0.0      324.54  0.46
smt2_mem_chase+int_alu   0.960  2.701  0.36  0.36  0.001  0.96      324.72  0.43
smt2_mem_stream+mem_chase 0.779  0.328  2.37  2.37  0.23  0.549      41.22  1295.44
smt2btb2_bsort+bsort     1.526  1.377  1.11  1.11  0.763  0.763      0.01  1.02
smt2btb2_dep_chain+dep_chain 1.776  0.887  2.00  2.00  0.888  0.888      0.0  0.24
smt2btb2_int_alu+int_alu 3.440  2.712  1.27  1.27  1.72  1.72      7.13  0.21
smt2pol_icount_int_alu+mem_chase 0.960  2.701  0.36  0.36  0.96  0.001      324.66  0.39
smt4_int_alu+dep_chain+mem_chase+branchy 2.197  0.777  2.83  2.83  0.881  0.5  0.461  0.355  0.35  63.19
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Table 8

SMT Throughput on the 4-Wide Core (Partitioned Buffers, Round-Robin Fetch and Commit)

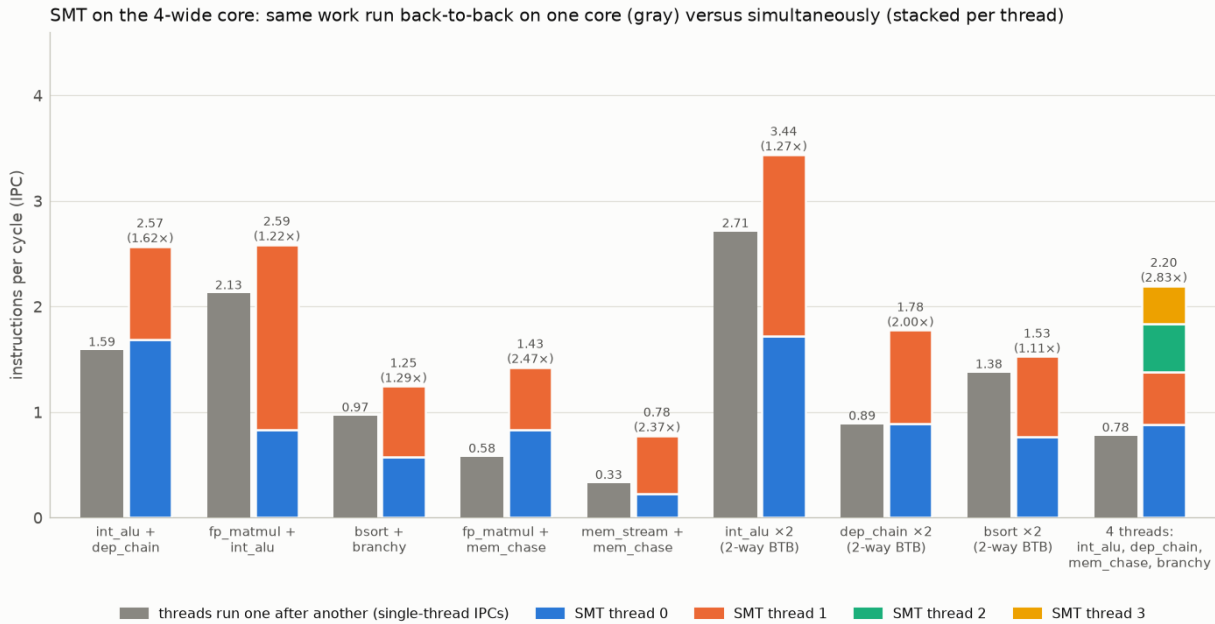
Threads	Per-thread IPC	Total IPC	Sequential IPC	Throughput speedup	Weighted speedup
int_alu + dep_chain	1.69 / 0.89	2.570	1.587	1.62×	1.62
fp_matmul + int_alu	0.83 / 1.76	2.588	2.125	1.22×	1.22
bsort + branchy	0.58 / 0.67	1.246	0.966	1.29×	1.29
fp_matmul + mem_chase	0.84 / 0.59	1.427	0.578	2.47×	2.47
mem_stream + mem_chase	0.23 / 0.55	0.779	0.328	2.37×	2.37
int_alu ×2 (1-way BTB)	1.44 / 1.44	2.876	2.712	1.06×	1.06

Threads	Per-thread IPC	Total IPC	Sequential IPC	Throughput speedup	Weighted speedup
int_alu ×2 (2-way BTB)	1.72 / 1.72	3.440	2.712	1.27×	1.27
dep_chain ×2 (1-way BTB)	0.57 / 0.57	1.142	0.887	1.29×	1.29
dep_chain ×2 (2-way BTB)	0.89 / 0.89	1.776	0.887	2.00×	2.00
bsort ×2 (1-way BTB)	0.69 / 0.69	1.377	1.377	1.00×	1.00
bsort ×2 (2-way BTB)	0.76 / 0.76	1.526	1.377	1.11×	1.11
int_alu + dep_chain + mem_chase + branchy	0.88 / 0.50 / 0.46 / 0.35	2.197	0.777	2.83×	2.83

Note. Sequential IPC = the same work run back to back on one core at the single-thread IPCs.
Weighted speedup = $\sum \text{IPC}_{i,\text{SMT}} / \text{IPC}_{i,\text{alone}}$.

Figure 35

Same Work Run Back to Back on One Core (Gray) Versus Simultaneously With SMT (Stacked per Thread)



Note. Labels give the SMT total IPC and the throughput speedup over sequential execution.

SMT improves throughput in every pairing, and by how much depends on what the threads leave idle. Two compute threads that each already use a good share of the 4-wide core gain modestly: fp_matmul with int_alu reaches 2.59 IPC, 1.22× the sequential 2.13, and two copies of int_alu reach 3.44 IPC (1.27×) once the BTB problem described below is fixed, because one int_alu alone already fills 2.7 of the 4 issue slots. Pairs in which one thread is stalled

most of the time gain the most: `int_alu` with `dep_chain` gives $1.62\times$ because the chain's μ ops wait in its half of the queue while `int_alu` uses the ALUs, two `dep_chain` copies give exactly $2.00\times$ since each chain is limited by its own latency and they interleave perfectly, and the memory-bound pairs give $2.4\text{--}2.5\times$ because one thread's cache misses overlap with the other's work (`fp_matmul` with `mem_chase` 1.43 IPC versus 0.58 sequential). Four threads together reach 2.20 IPC, $2.83\times$ the throughput of running them one after another. The weighted speedups equal the throughput speedups here because each thread received a fair share of the machine.

Resource Sharing and Contention Points

Table 9

How the Shared Pipeline Is Used: Buffer-Full Events per Thousand Instructions and Misprediction Rate, Alone Versus in SMT

Configuration	ROB full	IQ full	Store queue full	Cond. mispredicted
<code>int_alu</code> alone	40.5	11.8	0.2	0.39%
<code>dep_chain</code> alone	0.0	405.8	0.2	0.04%
<code>int_alu</code> + <code>dep_chain</code> (SMT)	4.1	101.2	0.3	0.05%
<code>fp_matmul</code> alone	42.2	0.0	21.5	1.96%
<code>fp_matmul</code> + <code>int_alu</code> (SMT)	13.9	20.6	13.7	1.74%
<code>mem_stream</code> alone	19.5	0.0	1014.2	0.01%
<code>mem_chase</code> alone	86.3	0.0	811.1	0.03%
<code>mem_stream</code> + <code>mem_chase</code> (SMT)	41.2	0.0	1295.4	0.02%
<code>int_alu</code> $\times 2$, 1-way BTB (SMT)	0.0	27.0	0.2	99.31%
<code>int_alu</code> $\times 2$, 2-way BTB (SMT)	7.1	27.0	0.2	0.22%

Note. With partitioned buffers each thread owns half of the ROB, IQ and queues. Round-robin fetch also halves the rate at which each partition fills, so per-instruction buffer-full events fall for `int_alu` and `fp_matmul` although their partitions are smaller; the throughput cost appears as shared fetch and issue bandwidth instead.

The counters show where threads collide. Partitioning halves each thread's ROB, yet `int_alu`'s ROB-full events per thousand instructions fall from 40 alone to 4–14 in SMT, because round-robin fetch feeds each thread only every other cycle, so the smaller partition fills less often; `int_alu`'s own IPC nevertheless drops from 2.71 to 1.69–1.76 because it now receives half

of the fetch, rename and issue bandwidth while the partner uses the other half. The instruction queue remains `dep_chain`'s wall and the store queue the memory programs' wall (Table 9); SMT does not relieve those, it overlaps them with another thread's progress. The functional units were never the bottleneck (`gem5`'s `fuBusyRate` stayed at zero) because the unit counts scale with width. Two contention points were less expected and are worth describing in detail.

A Shared, Thread-Tagged Branch Target Buffer

When two copies of the same program run together, the misprediction rate explodes: 99% of `int_alu`'s conditional branches and 67% of `dep_chain`'s are counted as mispredicted, IPC drops and the BTB hit ratio falls to 20–38% (Figure 36). The cause is that `gem5` tags each BTB entry with the thread that installed it and the default BTB is direct-mapped: both threads execute the same loop branch at the same address, so they evict each other's entry every iteration, the predicted-taken branch has no target, `gem5` falls back to “not taken”, and the branch is mispredicted (the `predTakenBTBMiss` counter equals the misprediction count). Making the BTB two-way set-associative with a new `--btb-assoc` option lets both threads' entries coexist: the hit ratio returns to 99–100%, mispredictions vanish, and the same-program pairs speed up by 20% (`int_alu`), 55% (`dep_chain`) and 11% (`bsort`) (Figure 37). Different programs do not collide because they occupy different addresses.

Figure 36

Two Copies of int_alu With the Default Direct-Mapped BTB Versus a 2-Way BTB: Cycles, BTB Hit Ratio, Predicted-Taken BTB Misses and Mispredictions

```

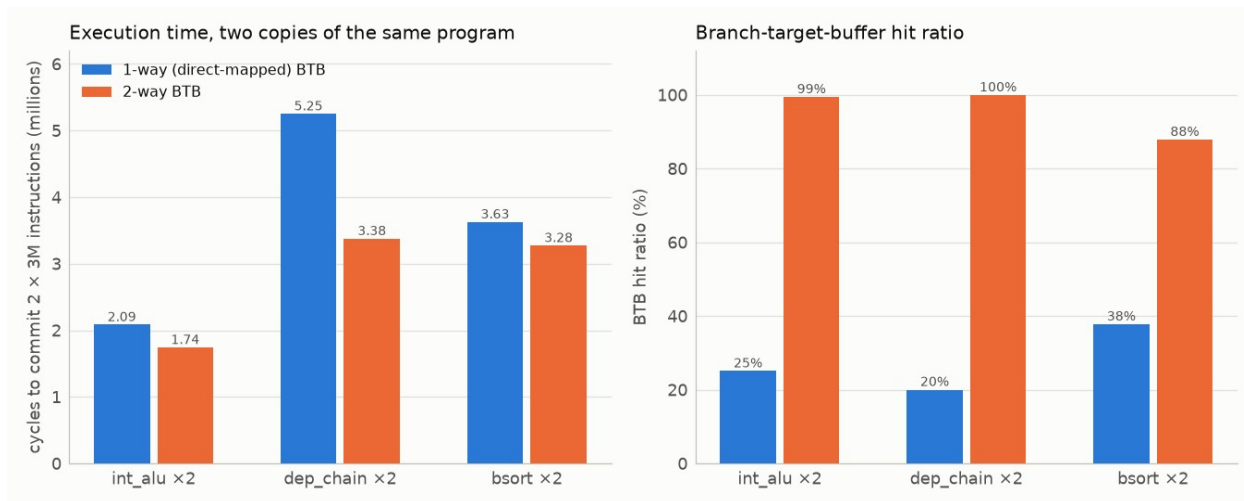
gem5-ilp-assignment4 -- -zsh -- 118x18

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % for r in smt2_int_alu+int_alu smt2btb2_int_alu+int_alu; do echo
"== "; grep -E '^system\.cpu\.(numCycles|branchPred\.(BTBHitRatio|predTakenBTBMiss|condIncorrect))\b' results/runs//st
ats.txt | cut -c1-110; done
==
grep: results/runs//stats.txt: No such file or directory
==
grep: results/runs//stats.txt: No such file or directory
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Figure 37

Effect of BTB Associativity on Same-Program SMT Pairs: Execution Time and BTB Hit Ratio



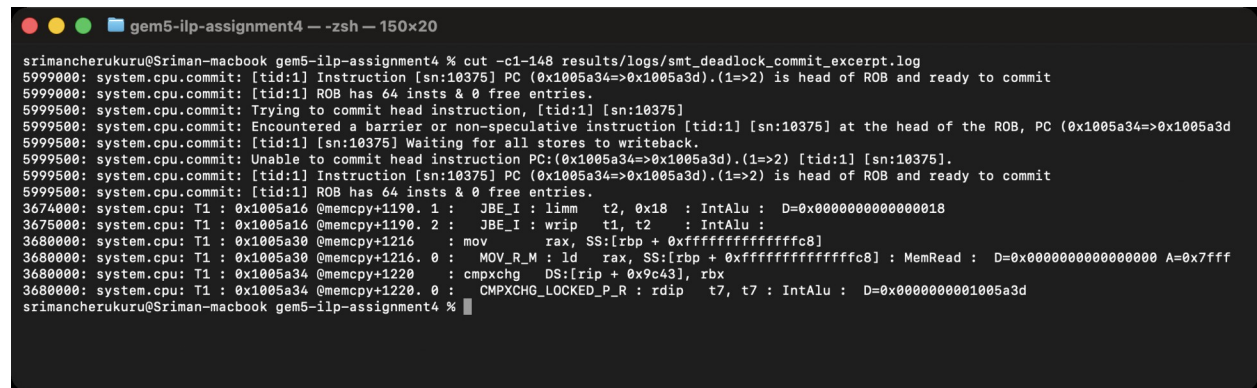
A Store-Writeback Deadlock in gem5's x86 SMT Model

Pairing mem_chase or mem_stream with a program that has no memory traffic (int_alu or dep_chain) produced runs in which the memory thread committed exactly 1,583 (or 1,484) instructions and then nothing, in either thread order and under both fetch policies, while the partner ran at a third of its usual speed. Tracing the first cycles (Figure 38) showed the stuck thread at the head of its ROB on a locked cmpxchg inside musl's malloc, with commit repeating

“waiting for all stores to writeback” forever: a store of that thread never wrote back, so the atomic instruction, which must wait for older stores, never committed. The same programs run correctly alone and next to `fp_matmul` or each other, so this is a defect in the interaction of `gem5`’s x86 total-store-order handling with SMT rather than a property of the workloads. I documented it and chose thread pairings that do not trigger it.

Figure 38

Commit-Stage Debug Output of the Stuck Thread and Its Last Executed Instructions: a Locked `cmpxchg` Waiting for a Store That Never Writes Back



```

srimancherukuru@Sriman-macbook: gem5-ilp-assignment4 % cut -c1-148 results/logs/smt_deadlock_commit_excerpt.log
5999000: system.cpu.commit: [tid:1] Instruction [sn:10375] PC (0x1005a34=>0x1005a3d).(1=>2) is head of ROB and ready to commit
5999000: system.cpu.commit: [tid:1] ROB has 64 insts & 0 free entries.
5999500: system.cpu.commit: Trying to commit head instruction, [tid:1] [sn:10375]
5999500: system.cpu.commit: Encountered a barrier or non-speculative instruction [tid:1] [sn:10375] at the head of the ROB, PC (0x1005a34=>0x1005a3d)
5999500: system.cpu.commit: [tid:1] [sn:10375] Waiting for all stores to writeback.
5999500: system.cpu.commit: Unable to commit head instruction PC:(0x1005a34=>0x1005a3d).(1=>2) [tid:1] [sn:10375].
5999500: system.cpu.commit: [tid:1] Instruction [sn:10375] PC (0x1005a34=>0x1005a3d).(1=>2) is head of ROB and ready to commit
5999500: system.cpu.commit: [tid:1] ROB has 64 insts & 0 free entries.
3674000: system.cpu: T1 : 0x1005a16 @memcpy+1190. 1 : JBE_I : limm t2, 0x18 : IntAlu : D=0x0000000000000018
3675000: system.cpu: T1 : 0x1005a16 @memcpy+1190. 2 : JBE_I : wrisp t1, t2 : IntAlu :
3680000: system.cpu: T1 : 0x1005a30 @memcpy+1216 : mov rax, SS:[rbp + 0xffffffffffffffc8]
3680000: system.cpu: T1 : 0x1005a30 @memcpy+1216. 0 : MOV_R_M : ld rax, SS:[rbp + 0xffffffffffffffc8] : MemRead : D=0x0000000000000000 A=0x7fff
3680000: system.cpu: T1 : 0x1005a34 @memcpy+1220 : cmpxchg DS:[rip + 0x9c43], rbx
3680000: system.cpu: T1 : 0x1005a34 @memcpy+1220. 0 : CMPXCHG_LOCKED_P_R : rdip t7, t7 : IntAlu : D=0x00000000001005a3d
srimancherukuru@Sriman-macbook: gem5-ilp-assignment4 %

```

Sharing Policies: Partitioned Versus Dynamic

`gem5` also offers Dynamic and Threshold sharing of the ROB and queues. Selecting either aborted the simulator immediately with a stack overflow whose backtrace was

LSQ::sqFull repeated hundreds of times: LSQ::sqFull(tid) calls sqFull(), which iterates over threads and calls sqFull(tid) again. I fixed the recursion by making the per-thread check compare the combined occupancy with the total number of entries (Figure 39, second commit in Figure 10). With that fix the Dynamic policy gets past initialization but then fails an assertion in ROB::insertInst because two threads are each allowed to fill the whole shared ROB (Figure 40); the policy has clearly not been maintained in this `gem5` version, so all SMT results above use the Partitioned policy, which is also what commercial two-thread cores do for their reorder buffers.

Figure 39

The LSQ Fix Committed on the gem5 Branch: LSQ::lqFull(tid) and sqFull(tid) No Longer Recurse Under the Dynamic Policy

```

gem5 --zsh -- 118x44

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % cd ~/git/gem5 && git log --oneline -3 && git diff HEAD~1 -- src/
cpu/o3/lq.cc | head -36
9ce3f337ba (HEAD -> ilp-assignment4) cpu/o3: fix infinite recursion in LSQ::lqFull/sqFull(tid) under the Dynamic SMT s
haring policy
36df987415 cpu/pred: add StaticBP, a static always-(not-)taken direction predictor for the ILP assignment
38716b67fb (origin/stable, stable) misc: add agentics-maintenance.yml to stable (#3459)
diff --git a/src/cpu/o3/lq.cc b/src/cpu/o3/lq.cc
index da04dc5931..35cd66ae8d 100644
--- a/src/cpu/o3/lq.cc
+++ b/src/cpu/o3/lq.cc
@@ -648,10 +648,13 @@ LSQ::lqFull()
    bool
    LSQ::lqFull(ThreadID tid)
    {
-   //@todo: Change to Calculate All Entries for
-   //Dynamic Policy
+   // Fix (ILP assignment): with the Dynamic sharing policy this used to
+   // call lqFull(), which calls lqFull(tid) again -> infinite recursion
+   // and a crash as soon as the policy is selected. A dynamically shared
+   // queue is full when the threads' combined occupancy reaches the total
+   // number of entries.
    if (lsqPolicy == SMTQueuePolicy::Dynamic)
-   return lqFull();
+   return numLoads() >= (int)LQEntries;
    else
        return thread[tid].lqFull();
    }
@@ -670,10 +673,10 @@ LSQ::sqFull()
    bool
    LSQ::sqFull(ThreadID tid)
    {
-   //@todo: Change to Calculate All Entries for
-   //Dynamic Policy
+   // Fix (ILP assignment): see lqFull(tid) -- the original sqFull() call
+   // recursed forever (LSQ::sqFull -> sqFull(tid) -> sqFull ...).
    if (lsqPolicy == SMTQueuePolicy::Dynamic)
-   return sqFull();
+   return numStores() >= (int)SQEntries;
    else
        return thread[tid].sqFull();
    }
srimancherukuru@Sriman-macbook gem5 %

```

Figure 40

After the LSQ Fix the Dynamic Policy Reaches a Second Defect: the ROB Over-Subscription Assertion

```

gem5-ilp-assignment4 --zsh -- 118x14

Program aborted at tick 1092000
0  gem5.opt 0x00000001011369b4 _ZN4gem515print_backtraceEv + 44
1  gem5.opt 0x000000010114a440 _ZN4gem512abortHandlerEi + 132
6  gem5.opt 0x000000010172b7c8 _ZN4gem52033ROB10insertInstERKNS_14RefCountingPtrINS0_7Dy
7  gem5.opt 0x00000001007fb700 _ZN4gem52033ROB10insertInstERKNS_14RefCountingPtrINS0_7Dy
8  gem5.opt 0x000000010077eb98 _ZN4gem52036Commit8getInstsEv + 372
9  gem5.opt 0x000000010077d72c _ZN4gem52036Commit6commitEv + 2372
10 gem5.opt 0x000000010077ca1c _ZN4gem52036Commit4tickEv + 424
11 gem5.opt 0x0000000100786514 _ZN4gem52033CPU4tickEv + 160
12 gem5.opt 0x0000000101142e10 _ZN4gem510EventQueue10serviceOneEv + 244
13 gem5.opt 0x000000010115d5b0 _ZN4gem59doSimLoopEPNS_10EventQueueE + 320
14 gem5.opt 0x000000010115cee0 _ZN4gem58simulateEy + 376
15 gem5.opt 0x000000010054fb94 _ZZN8pybind1112cpp_function10initializeIRPFPN4gem522Globa
srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Does SMT significantly improve the number of instructions completed per cycle? On this core it does: 22–62% more throughput for pairs of compute-bound threads, 2.4–2.5× for

memory-bound pairs and $2.8\times$ with four mixed threads, all with a fair split between threads. The gains come from filling idle issue slots and overlapping stalls, so they are largest exactly where the single-thread ILP techniques of Sections 4 and 5 gave up: the dependency chain and the pointer chase.

Key Questions

How Do the ILP Techniques Interact?

The experiments show that the techniques multiply rather than add. Width without prediction is wasted: on `bsort` the tournament predictor is worth 31% on the 1-wide core and 96% on the 4-wide core, and the fraction of fetched μ ops that are squashed rises from 8% to 52% between the 1-wide and 8-wide cores. Width without independent work is also wasted, and SMT is the technique that recovers it: `dep_chain` stops scaling at 2-wide, but two `dep_chain` threads on the 4-wide core reach $2.0\times$ the throughput. The memory system interacts with all of them: a larger window and load queue raise `mem_stream`'s throughput by overlapping misses, but they also raise every instruction's latency, and `mem_chase` is immune to every single-thread technique and only helped by SMT. Even the predictor and SMT interact, through the shared BTB.

What Are the Limitations of These Techniques?

Each technique runs into a limit set by the program rather than the hardware. Multiple issue is bounded by the length of the critical dependence chain (`dep_chain`: 0.89 IPC at any width), branch prediction by the information content of the branch outcomes (`branchy`: 15% mispredictions with every predictor), and deeper windows by memory latency (`mem_chase`: 943 cycles of latency per μ op for 0.31 IPC). SMT does not remove any of these limits; it hides them by finding work elsewhere, and it introduces its own: partitioned buffers halve what each thread

gets, shared structures such as the BTB can thrash, and the simulator itself showed how delicate the shared machinery is (the store-writeback deadlock and the unmaintained Dynamic policy).

How Can Complexity Be Balanced Against Benefit?

The sweep gives a concrete answer for this design. Going from 1 to 2 wide roughly doubles the buffers and functional units and gives 1.6–1.8× on the ILP-rich programs; going from 4 to 8 wide doubles them again for 1.1–1.4×, and for the dependency- and latency-bound programs for nothing, while every instruction’s latency and every misprediction’s cost keep growing. The 4-wide core is where most of the benefit of width is captured, which is why real cores of this class are four to six wide and spend their remaining transistors on prediction, caches and SMT instead. Branch prediction is the cheapest technique per unit of benefit (a few kilobytes of tables for up to 2.5× on bsort), and SMT is the most effective way to use the slots that width leaves empty, at the cost of per-thread fairness and shared-structure design care.

Troubleshooting and Issue Log

Table 10 lists every problem met while doing the assignment, how it was diagnosed and how it was resolved; three of them are illustrated in Figures 41 to 43.

Table 10
Issues Encountered, Their Diagnosis and Resolution

Issue	Symptom	Cause and fix
Instruction-queue size parameter	AttributeError: Invalid assignment for Class X86O3CPU with parameter numIQEntries	In v25.1 the IQ moved into an IQUnit object; the size is <code>instQueues=[IQUnit(numEntries=...)]</code> .
FP kernel crashed in gem5	panic: Tried to write unmapped address 0x7fffffff000 at every width and with every predictor	musl’s <code>printf(“%f”)</code> uses x87 long-double code that gem5’s x86 model executes incorrectly; printing the checksum as a scaled integer avoids the path.
MinorCPU unusable on x86	Stock or tuned in-order MinorCPU ran <code>int_alu</code> at 24 cycles per instruction	The MinorTrace/MinorExecute trace shows the execute stage discarding a stream of “(invalid)” wrong-path instructions every cycle after each branch; the model’s x86 support is not usable here, so the 1-wide O3 core is the basic pipeline.
Empty field in the run matrix	Only the four traced demo runs completed; other runs failed instantly with <code>argparse</code> errors	<code>xargs -0</code> drops an empty field, shifting the arguments; the matrix now writes NONE for “no gem5 options” and the runner strips it.

Issue	Symptom	Cause and fix
Same-program SMT threads mispredict 99%	condIncorrect $\approx$ predTakenBTBMiss; BTB hit ratio 20–38%	Thread-tagged, direct-mapped BTB entries thrash; added --btb-assoc and used a 2-way BTB for those pairs (Section 6).
SMT thread starves	One thread committed exactly 1,583 (mem_chase) or 1,484 (mem_stream) instructions, then nothing	Store-writeback deadlock at a locked cmpxchg in malloc (gem5 x86 TSO + SMT); documented, and pairings that trigger it excluded (Section 6).
Dynamic/Threshold sharing policy aborts	Immediate stack overflow, backtrace of LSQ::sqFull repeated	Infinite recursion in LSQ::lqFull/sqFull(tid); fixed in lsq.cc, after which a ROB over-subscription assertion shows the policy is unmaintained; Partitioned policy used.
Trace throughput above stats IPC	pipestats reported 1.56 “IPC” for branchy where stats.txt said 0.77	O3PipeView records μ ops, not instructions; the latency table reports μ op throughput and divides by the measured μ op ratio to cross-check against IPC.
Latency windows at trace edges	μ ops fetched before or retiring after a tick window have incomplete records	pipestats trims the first and last 10% of fetched μ ops in the window before measuring.

Note. All logs referenced here are in results/logs/ of the repository.

Figure 41

The First Run of the Configuration Script: the IQ-Size Parameter Had Moved Into IQUnit in gem5 v25.1

```

gem5-ilp-assignment4 — zsh — 118x16

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 % head -10 results/logs/config_error_numIQEntries.log
AttributeError: Invalid assignment for Class X86O3CPU with parameter numIQEntries

At:
src/python/m5/SimObject.py(934): __setattr__
configs/ilp_config.py(127): make_o3
configs/ilp_config.py(171): <module>
src/python/m5/main.py(687): main
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

srimancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```


Figure 42

The FP Kernel's Panic Before the Checksum Was Changed to an Integer: an Unmapped Write Above the Stack From musl's x87 printf Path

```

gem5-ilp-assignment4 — -zsh — 118x26

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % grep -n -B3 -A4 'panic' results/logs/fp_matmul_x87_printf_panic.
log | head -24 | cut -c1-116
22-build/X86/arch/x86/generated/exec-ns.cc.inc:27: warn: instruction 'fwait' unimplemented
23-build/X86/arch/x86/generated/exec-ns.cc.inc:27: warn: instruction 'fimul' unimplemented
24-build/X86/arch/x86/generated/exec-ns.cc.inc:27: warn: instruction 'fimul' unimplemented
25:src/arch/x86/faults.cc:169: panic: Tried to write unmapped address 0x7fffffff000.
26-PC: (0x10036b7=>0x10036ba).(0=>1), Instr:  MOV_M_R : st  eax, DS:[r9]
27-Memory Usage: 437402416 KBytes
28-Program aborted at tick 490386500
29----- BEGIN LIBC BACKTRACE -----
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Figure 43

Why the In-Order MinorCPU Was Dropped: Its Execute Stage Discards “(invalid)” Wrong-Path x86 Instructions Every Cycle

```

gem5-ilp-assignment4 — -zsh — 160x26

srinancherukuru@Sriman-macbook gem5-ilp-assignment4 % head -22 results/logs/minorcpu_x86_trace_excerpt.log | cut -c1-158
1000000000: system.cpu.execute: Attempting to issue [tid:0]
1000000000: system.cpu.execute: Discarding inst: 0/28.1/81/505.3996755 pc: 0x1005562 (panic) as its stream state was unexpected, expected: 29
1000000000: system.cpu.execute: MinorInst: id=0/28.1/81/505.3996755 addr=0x1005562 inst="(invalid)" class=No_OpClass flags="IsInteger IsFloating IsVector IsVe
1000000000: system.cpu.execute: Stepping to next inst inputIndex: 1
1000000000: system.cpu.execute: Discarding inst: 0/28.1/81/505.3996756 pc: 0x1005562 (panic) as its stream state was unexpected, expected: 29
1000000000: system.cpu.execute: MinorInst: id=0/28.1/81/505.3996756 addr=0x1005562 inst="(invalid)" class=No_OpClass flags="IsInteger IsFloating IsVector IsVe
1000000000: system.cpu.execute: Stepping to next inst inputIndex: 2
1000000000: system.cpu.execute: Wrapping
1000000500: system.cpu.execute: Attempting to issue [tid:0]
1000000500: system.cpu.execute: Discarding inst: 0/28.1/81/505.3996757 pc: 0x1005562 (panic) as its stream state was unexpected, expected: 29
1000000500: system.cpu.execute: MinorInst: id=0/28.1/81/505.3996757 addr=0x1005562 inst="(invalid)" class=No_OpClass flags="IsInteger IsFloating IsVector IsVe
1000000500: system.cpu.execute: Stepping to next inst inputIndex: 1
1000000500: system.cpu.execute: Discarding inst: 0/28.1/81/505.3996758 pc: 0x1005562 (panic) as its stream state was unexpected, expected: 29
1000000500: system.cpu.execute: MinorInst: id=0/28.1/81/505.3996758 addr=0x1005562 inst="(invalid)" class=No_OpClass flags="IsInteger IsFloating IsVector IsVe
1000000500: system.cpu.execute: Stepping to next inst inputIndex: 2
1000000500: system.cpu.execute: Wrapping
1000001000: system.cpu.execute: Attempting to issue [tid:0]
1000001000: system.cpu.execute: Discarding inst: 0/28.1/81/505.3996759 pc: 0x1005562 (panic) as its stream state was unexpected, expected: 29
1000001000: system.cpu.execute: MinorInst: id=0/28.1/81/505.3996759 addr=0x1005562 inst="(invalid)" class=No_OpClass flags="IsInteger IsFloating IsVector IsVe
1000001000: system.cpu.execute: Stepping to next inst inputIndex: 1
1000001000: system.cpu.execute: Discarding inst: 0/28.1/81/505.3996760 pc: 0x1005562 (panic) as its stream state was unexpected, expected: 29
1000001000: system.cpu.execute: MinorInst: id=0/28.1/81/505.3996760 addr=0x1005562 inst="(invalid)" class=No_OpClass flags="IsInteger IsFloating IsVector IsVe
srinancherukuru@Sriman-macbook gem5-ilp-assignment4 %

```

Conclusion

Starting from the gem5 build of the earlier assignments, this work turned one configuration script into 98 simulations that measure the classical ILP techniques on a single x86 core. The basic 1-wide pipeline needs seven cycles per μop and delivers at most one μop per

cycle; the pipeline viewer and Gantt charts make visible that a dependency chain waits in the instruction queue while independent lanes flow through untouched. Widening the core to 8 issue slots gives $4.3\times$ on ALU-rich code, $3.3\times$ on floating point, and almost nothing on a serial chain or a pointer chase, while every instruction's latency grows with the window. Branch prediction is what makes width usable: on bubble sort TAGE delivers 2.5 times the IPC of a static predictor at 4-wide, and the value of prediction triples between the 1-wide and 4-wide cores. SMT recovers the slots the other techniques leave empty, $1.2\text{--}1.6\times$ for compute pairs and $2.4\text{--}2.8\times$ for memory-bound and mixed sets, provided shared structures such as the BTB are designed for it. Along the way the assignment required real engineering: a new predictor class, a bug fix in gem5's SMT load/store queue, a workaround for an x87 emulation gap, and the diagnosis of a store-writeback deadlock, all of which are recorded in the repository at

<https://github.com/srimancherukuru/gem5-ilp-assignment4>.

References

- Binkert, N., Beckmann, B., Black, G., Reinhardt, S. K., Saidi, A., Basu, A., Hestness, J., Hower, D. R., Krishna, T., Sardashti, S., Sen, R., Sewell, K., Shoaib, M., Vaish, N., Hill, M. D., & Wood, D. A. (2011). The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2), 1–7. <https://doi.org/10.1145/2024716.2024718>
- Cherukuru, S. (2026). *gem5-ilp-assignment4: Exploring instruction-level parallelism in gem5* [Source code and data]. GitHub. <https://github.com/srimancherukuru/gem5-ilp-assignment4>
- gem5 Project. (n.d.). *Visualization*. gem5 documentation. Retrieved September 13, 2026, from https://www.gem5.org/documentation/general_docs/cpu_models/visualization/
- gem5 Project. (2026). *gem5 source code* (Version 25.1.0.1) [Computer software]. GitHub. <https://github.com/gem5/gem5>
- Hennessy, J. L., & Patterson, D. A. (2019). *Computer architecture: A quantitative approach* (6th ed.). Morgan Kaufmann.
- Lowe-Power, J., Ahmad, A. M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., Black, G., Bloom, G., Bruce, B. R., Carvalho, D. R., Castrillon, J., Chen, L., Derumigny, N., Diestelhorst, S., Elsasser, W., . . . Zulian, É. F. (2020). *The gem5 simulator: Version 20.0+*. arXiv. <https://doi.org/10.48550/arXiv.2007.03152>
- McFarling, S. (1993). *Combining branch predictors* (WRL Technical Note TN-36). Digital Equipment Corporation Western Research Laboratory.
- Seznec, A., & Michaud, P. (2006). A case for (partially) TAgged GEometric history length branch prediction. *Journal of Instruction-Level Parallelism*, 8, 1–23.

Tullsen, D. M., Eggers, S. J., & Levy, H. M. (1995). Simultaneous multithreading: Maximizing on-chip parallelism. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA '95)* (pp. 392–403). ACM.

<https://doi.org/10.1145/223982.224449>

Appendix A

The gem5 Configuration Script (configs/ilp_config.py)

```
"""
ilp_config.py -- gem5 syscall-emulation (SE) configuration for
Assignment 4, "Exploring Instruction-Level Parallelism", by Sriman Cherukuru.

One script drives every experiment in Part 2:

--cpu o3      --width 1      scalar (single-issue) pipeline, the baseline
--cpu o3      --width 2|4|8  superscalar (multiple-issue) pipelines
--cpu minor   a classic in-order pipeline for comparison
--bp static-nt|static-t|local|tournament|tage
branch-direction predictor (static-* = the
StaticBP class added to gem5 for this study)
--cmd BIN[:ARG] workload; give --cmd twice or more for SMT
(the O3 core then gets one hardware thread
per program and they share the pipeline)

Example:
    build/X86/gem5.opt --outdir=m5out/ss4 configs/ilp_config.py \
    --cpu o3 --width 4 --bp tournament --cmd bin/Int_alu:60000
"""
import argparse
import os

import m5
from m5.objects import *
import m5.objects as objs

# ----- options
parser = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
parser.add_argument("--cpu", choices=["o3", "minor"], default="o3",
                    help="o3 = out-of-order (default), minor = in-order")
parser.add_argument("--width", type=int, default=1, choices=[1, 2, 4, 8],
                    help="issue width; applied to every O3 stage")
parser.add_argument("--bp", default="tournament",
                    choices=["static-nt", "static-t", "local", "tournament", "tage"],
                    help="conditional branch direction predictor")
parser.add_argument("--cmd", action="append", required=True, metavar="BIN[:ARG[:ARG]]",
                    help="program to run; repeat for SMT (one HW thread each)")
parser.add_argument("--clock", default="2GHz", help="CPU clock (default 2GHz)")
parser.add_argument("--rob", type=int, help="override reorder-buffer entries")
parser.add_argument("--iq", type=int, help="override instruction-queue entries")
parser.add_argument("--lsq", type=int, help="override load- and store-queue entries")
parser.add_argument("--smt-fetch", default="RoundRobin",
                    choices=["RoundRobin", "Branch", "IQCount", "LSQCount"],
                    help="SMT fetch policy (O3, >1 thread)")
parser.add_argument("--smt-share", default="Partitioned",
                    choices=["Dynamic", "Partitioned", "Threshold"],
                    help="SMT ROB/IQ/LSQ sharing policy (O3, >1 thread)")
parser.add_argument("--btb-assoc", type=int, default=1,
                    help="BTB associativity (2+ lets SMT threads that run the same code share sets)")
parser.add_argument("--minor-stock", action="store_true",
                    help="use gem5's stock MinorCPU parameters (no single-issue tweaks)")
parser.add_argument("--maxinsts", type=int, default=0,
                    help="stop after this many instructions on any thread")
args = parser.parse_args()

# ----- the caches
# A two-level hierarchy that is the same for every experiment, so only the
# core changes between runs (latencies follow configs/learning_gem5).
class L1Cache(Cache):
    assoc = 4
    tag_latency = 2
    data_latency = 2
    response_latency = 2
    mshrs = 8
    tgts_per_mshr = 20

class L1ICache(L1Cache):
    size = "32KiB"

class L1DCache(L1Cache):
    size = "64KiB"

class L2Cache(Cache):
```

```

size = "512KiB"
assoc = 8
tag_latency = 10
data_latency = 10
response_latency = 5
mshrs = 32
tgts_per_mshr = 16

# ----- branch predictors
def make_bp(kind):
    """Return a BranchPredictor unit whose *direction* predictor is `kind`.
    The BTB, return-address stack and indirect predictor stay at gem5's
    defaults so only the direction-prediction policy changes."""
    if kind == "static-nt":
        cond = StaticBP(predictTaken=False)      # always fall through
    elif kind == "static-t":
        cond = StaticBP(predictTaken=True)       # always take
    elif kind == "local":
        cond = LocalBP(localPredictorSize=2048, localCtrBits=2)  # 2-bit counters
    elif kind == "tournament":
        cond = TournamentBP()                   # local + global + chooser
    else:
        cond = TAGE()                           # state-of-the-art tagged geometric
    bp = BranchPredictor(conditionalBranchPred=cond)
    if args.btb_assoc > 1:
        bp.btb = SimpleBTB(associativity=args.btb_assoc)
    return bp

# ----- functional units
# Functional-unit pool scaled with the issue width: `w` integer ALUs, and
# about w/2 of each of the other unit types (never fewer than one).
IntMulDivCls = getattr(objs, "X86IntMultDiv", IntMultDiv)

def make_fu_pool(w):
    half = max(1, w // 2)
    return FUPool(FUList=[
        IntALU(count=w),
        IntMulDivCls(count=half),
        FP_ALU(count=half),
        FP_MultDiv(count=half),
        SIMD_Unit(count=half),
        RdWrPort(count=half),          # load/store ports
        PredALU(count=1),
        Matrix_Unit(count=1),
        System_Unit(count=1),
    ])

# Buffer sizes that grow with the width (ROB, IQ, LQ/SQ entries).
SIZES = {1: (32, 16, 8), 2: (64, 32, 16), 4: (128, 64, 32), 8: (192, 96, 48)}

def make_o3(nthreads):
    w = args.width
    rob, iq, lsq = SIZES[w]
    cpu = X86O3CPU(numThreads=nthreads)
    for p in ("fetchWidth", "decodeWidth", "renameWidth", "dispatchWidth",
              "issueWidth", "wbWidth", "commitWidth", "squashWidth"):
        setattr(cpu, p, w)
    cpu.numROBEntries = args.rob or rob
    cpu.instQueues = [IQUnit(numEntries=args.iq or iq,          # IQ lives in IQUnit in gem5 v25
                             smtIQPolicy=args.smt_share)]
    cpu.LQEntries = args.lsq or lsq
    cpu.SQEntries = args.lsq or lsq
    cpu.fuPool = make_fu_pool(w)
    cpu.branchPred = make_bp(args.bp)
    if nthreads > 1:
        # SMT policies
        cpu.smtFetchPolicy = args.smt_fetch
        cpu.smtROBPolicy = args.smt_share
        cpu.smtLSQPolicy = args.smt_share
        cpu.smtNumFetchingThreads = 1
    return cpu

def make_minor(nthreads):
    """Single-issue in-order pipeline: Fetch1 -> Fetch2 -> Decode -> Execute."""
    cpu = X86MinorCPU(numThreads=nthreads)
    cpu.branchPred = make_bp(args.bp)
    if args.minor_stock:
        return cpu
    cpu.fetch1FetchLimit = 1
    cpu.fetch2InputBufferSize = 2
    cpu.decodeInputWidth = 1
    cpu.decodeInputBufferSize = 3
    cpu.executeInputWidth = 1
    cpu.executeIssueLimit = 1
    cpu.executeMemoryIssueLimit = 1
    cpu.executeCommitLimit = 1
    cpu.executeMemoryCommitLimit = 1

```

```

    cpu.executeInputBufferSize = 7
    return cpu

# ----- workloads
procs = []
for idx, spec in enumerate(args.cmd):
    parts = spec.split(":")
    binary = os.path.abspath(parts[0])
    procs.append(Process(pid=100 + idx, cmd=[binary] + parts[1:]))
nthreads = len(procs)

# ----- system
system = System()
system.clk_domain = SrcClockDomain(clock=args.clock, voltage_domain=VoltageDomain())
system.mem_mode = "timing"
system.mem_ranges = [AddrRange("1GiB")]
if nthreads > 1:
    system.multi_thread = True

system.cpu = make_o3(nthreads) if args.cpu == "o3" else make_minor(nthreads)
if args.maxinsts:
    system.cpu.max_insts_any_thread = args.maxinsts

# L1 caches on the core, a shared L2, then the memory bus and DDR3.
system.cpu.icache = L1ICache()
system.cpu.dcache = L1DCache()
system.cpu.icache.cpu_side = system.cpu.icache_port
system.cpu.dcache.cpu_side = system.cpu.dcache_port
system.l2bus = L2XBar()
system.cpu.icache.mem_side = system.l2bus.cpu_side_ports
system.cpu.dcache.mem_side = system.l2bus.cpu_side_ports
system.l2cache = L2Cache()
system.l2cache.cpu_side = system.l2bus.mem_side_ports
system.membus = SystemXBar()
system.l2cache.mem_side = system.membus.cpu_side_ports
system.mem_ctrl = MemCtrl()
system.mem_ctrl.dram = DDR3_1600_8x8(range=system.mem_ranges[0])
system.mem_ctrl.port = system.membus.mem_side_ports
system.system_port = system.membus.cpu_side_ports

# x86 needs one interrupt controller per hardware thread, each with 3 ports.
system.cpu.createInterruptController()
for i in range(nthreads):
    system.cpu.interrupts[i].pio = system.membus.mem_side_ports
    system.cpu.interrupts[i].int_requestor = system.membus.cpu_side_ports
    system.cpu.interrupts[i].int_responder = system.membus.mem_side_ports

system.workload = SEWorkload.init_compatible(procs[0].cmd[0])
system.cpu.workload = procs if nthreads > 1 else procs[0]
system.cpu.createThreads()

# ----- simulate
root = Root(full_system=False, system=system)
m5.instantiate()
print(f"[ilp_config] cpu={args.cpu} width={args.width} bp={args.bp} "
      f"threads={nthreads} cmds={args.cmd}")
exit_event = m5.simulate()
ticks = m5.curTick()
print(f"[ilp_config] exiting @ tick {ticks} because {exit_event.getCause()}")

```

Appendix B

The StaticBP Predictor Added to gem5 (src/cpu/pred/static_bp.hh and static_bp.cc)

```

/*
 * StaticBP -- a static (non-learning) conditional branch direction
 * predictor, written for the ILP assignment (Assignment 4) as the
 * "no dynamic branch prediction" baseline. Every conditional branch is
 * predicted with the same fixed direction (always-not-taken by default,
 * or always-taken), so the predictor never adapts to program behaviour.
 * It keeps no per-branch state, therefore bp_history is always NULL,
 * exactly like LocalBP (cpu/pred/2bit_local.hh), which served as the
 * template for this class.
 */

#ifndef __CPU_PRED_STATIC_BP_HH__
#define __CPU_PRED_STATIC_BP_HH__

#include "base/types.hh"
#include "cpu/pred/conditional.hh"
#include "params/StaticBP.hh"

namespace gem5
{
    namespace branch_prediction
    {
        class StaticBP : public ConditionalPredictor
        {
        public:
            StaticBP(const StaticBPParams &params);

            bool lookup(ThreadID tid, Addr pc, void * &bp_history) override;

            void branchPlaceholder(ThreadID tid, Addr pc, bool uncond,
                                   void * &bp_history) override;

            void updateHistories(ThreadID tid, Addr pc, bool uncond, bool taken,
                                  Addr target, const StaticInstPtr &inst,
                                  void * &bp_history) override;

            void update(ThreadID tid, Addr pc, bool taken,
                        void * &bp_history, bool squashed,
                        const StaticInstPtr &inst, Addr target) override;

            void squash(ThreadID tid, void * &bp_history) override
            { assert(bp_history == NULL); }

        private:
            /** The one direction every conditional branch is predicted with. */
            const bool predictTaken;
        };
    } // namespace branch_prediction
} // namespace gem5

#endif // __CPU_PRED_STATIC_BP_HH__

/*
 * StaticBP -- static direction predictor (see static_bp.hh).
 */

#include "cpu/pred/static_bp.hh"

#include "base/trace.hh"
#include "debug/Fetch.hh"

namespace gem5
{

```



```

namespace branch_prediction
{

StaticBP::StaticBP(const StaticBPParams &params)
    : ConditionalPredictor(params),
      predictTaken(params.predictTaken)
{
    DPRINTF(Fetch, "StaticBP: every conditional branch is predicted %s\n",
            predictTaken ? "taken" : "not-taken");
}

void
StaticBP::branchPlaceholder(ThreadID tid, Addr pc, bool uncond,
                           void * &bp_history)
{
    // No predictor state, so there is no history object to create.
}

void
StaticBP::updateHistories(ThreadID tid, Addr pc, bool uncond, bool taken,
                          Addr target, const StaticInstPtr &inst,
                          void * &bp_history)
{
    // A static predictor keeps no history.
}

bool
StaticBP::lookup(ThreadID tid, Addr pc, void * &bp_history)
{
    DPRINTF(Fetch, "StaticBP: branch %#x predicted %s\n", pc,
            predictTaken ? "taken" : "not-taken");
    return predictTaken;
}

void
StaticBP::update(ThreadID tid, Addr pc, bool taken, void * &bp_history,
                 bool squashed, const StaticInstPtr &inst, Addr target)
{
    // Nothing is learned from the outcome: the prediction never changes.
    assert(bp_history == NULL);
}

} // namespace branch_prediction
} // namespace gem5

```

Appendix C

Workload Sources (workloads/)

common.h

```
/* Shared helpers for the Assignment 4 ILP workloads (Sriman Cherukuru).
 * Every program is a small, self-checking kernel: it takes an optional
 * iteration count, runs, and prints a checksum so correct execution
 * inside gem5 can be verified against the native run. */
#ifndef ILP_COMMON_H
#define ILP_COMMON_H
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
static inline uint64_t lcg_next(uint64_t *s) { /* fixed-seed PRNG */
    *s = *s * 6364136223846793005ULL + 1442695040888963407ULL;
    return *s >> 33;
}
static inline long arg_or(int argc, char **argv, long dflt) {
    return argc > 1 ? atol(argv[1]) : dflt;
}
#endif
```

int_alu.c

```
/* int_alu: eight independent xorshift64 lanes -> integer-ALU bound with
 * plenty of ILP (8 independent dependency chains per iteration). */
#include "common.h"
static inline uint64_t xs(uint64_t x) { x ^= x << 13; x ^= x >> 7; x ^= x << 17; return x; }
int main(int argc, char **argv) {
    long n = arg_or(argc, argv, 60000);
    uint64_t a = 1, b = 2, c = 3, d = 4, e = 5, f = 6, g = 7, h = 8;
    for (long i = 0; i < n; i++) {
        a = xs(a); b = xs(b); c = xs(c); d = xs(d);
        e = xs(e); f = xs(f); g = xs(g); h = xs(h);
    }
    printf("int_alu: n=%ld checksum=%016llx\n", n,
        (unsigned long long)(a ^ b ^ c ^ d ^ e ^ f ^ g ^ h));
    return 0;
}
```

dep_chain.c

```
/* dep_chain: ONE serial dependency chain (LCG multiply-add + shift/xor).
 * Each iteration needs the previous result, so ILP is ~1 no matter how
 * wide the machine is: the ILP-limit case. */
#include "common.h"
int main(int argc, char **argv) {
    long n = arg_or(argc, argv, 500000);
    uint64_t x = 1;
    for (long i = 0; i < n; i++) {
        x = x * 6364136223846793005ULL + 1442695040888963407ULL;
        x ^= x >> 29;
    }
    printf("dep_chain: n=%ld checksum=%016llx\n", n, (unsigned long long)x);
    return 0;
}
```

fp_matmul.c

```
/* fp_matmul: 48x48 double-precision matrix multiply (i-k-j order,
 * stride-1 inner loop), repeated R times -> floating-point multiply/add
 * with independent accumulators across j; working set ~55 KiB. */
#include "common.h"
#define N 48
static double A[N][N], B[N][N], C[N][N];
int main(int argc, char **argv) {
    long reps = arg_or(argc, argv, 4);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = (i * 7 + j * 3) % 11 / 10.0 + 0.5;
            B[i][j] = (i * 5 + j * 13) % 17 / 20.0 + 0.25;
            C[i][j] = 0.0;
        }
}
```

```

    for (long r = 0; r < reps; r++)
        for (int i = 0; i < N; i++)
            for (int k = 0; k < N; k++) {
                double a = A[i][k];
                for (int j = 0; j < N; j++)
                    C[i][j] += a * B[k][j];
            }
    double sum = 0.0;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) sum += C[i][j];
    /* print as a scaled integer: musl's %f formatting uses x87 long-double
     * code that gem5's x86 model does not emulate correctly (see report). */
    printf("fp_matmul: reps=%ld checksum=%lld\n", reps, (long long)(sum * 1000.0));
    return 0;
}

```

mem_stream.c

```

/* mem_stream: sequential sweep over a 4 MiB array (far larger than the
 * caches), R read passes -> memory-bandwidth/latency bound streaming. */
#include "common.h"
#define WORDS (512 * 1024) /* 512 Ki x 8 B = 4 MiB */
int main(int argc, char **argv) {
    long passes = arg_or(argc, argv, 3);
    uint64_t *a = malloc(WORDS * sizeof(uint64_t));
    if (!a) return 1;
    for (long i = 0; i < WORDS; i++) a[i] = (uint64_t)i * 2654435761ULL;
    uint64_t sum = 0;
    for (long p = 0; p < passes; p++)
        for (long i = 0; i < WORDS; i++) sum += a[i];
    printf("mem_stream: passes=%ld checksum=%016llx\n", passes, (unsigned long long)sum);
    free(a);
    return 0;
}

```

mem_chase.c

```

/* mem_chase: pointer chasing through a 2 MiB array where every hop lands
 * on a new cache line and page (stride 4097 words). Each load depends
 * on the previous one -> serialized cache misses, no MLP: latency bound. */
#include "common.h"
#define M (512 * 1024) /* 512 Ki x 4 B = 2 MiB */
#define STRIDE 4097 /* odd => single cycle over M */
int main(int argc, char **argv) {
    long hops = arg_or(argc, argv, 300000);
    uint32_t *next = malloc(M * sizeof(uint32_t));
    if (!next) return 1;
    for (long i = 0; i < M; i++) next[i] = (uint32_t)((i + STRIDE) % M);
    uint32_t p = 0;
    for (long h = 0; h < hops; h++) p = next[p];
    printf("mem_chase: hops=%ld final=%u\n", hops, p);
    free(next);
    return 0;
}

```

branchy.c

```

/* branchy: data-dependent conditional branches on pseudo-random data.
 * The two branch bodies have different memory side effects so the
 * compiler cannot turn them into branch-free cmov code. About half of
 * the branches are taken at random -> the branch-predictor stress test. */
#include "common.h"
#define SZ 16384
static uint32_t data[SZ];
static uint32_t hist[256];
int main(int argc, char **argv) {
    long passes = arg_or(argc, argv, 40);
    uint64_t seed = 12345;
    for (int i = 0; i < SZ; i++) data[i] = (uint32_t)lcg_next(&seed);
    uint64_t s = 0, t = 0;
    for (long p = 0; p < passes; p++)
        for (int i = 0; i < SZ; i++) {
            uint32_t x = data[i];
            if (x & 1) { /* random 50/50 branch */
                s += x;
                hist[x & 255]++;
            } else {
                t ^= x;
            }
        }
}

```

```

        if (x > 0x60000000u) s += 3;    /* random ~25/75 branch */
    }
    uint64_t hs = 0;
    for (int i = 0; i < 256; i++) hs += hist[i];
    printf("branchy: passes=%ld checksum=%016llx hist=%llu\n", passes,
        (unsigned long long)(s ^ t), (unsigned long long)hs);
    return 0;
}

```

bsort.c

```

/* bsort: bubble sort of an 800-element pseudo-random array: a mixed
 * integer/memory/branch kernel whose comparison branches start out
 * unpredictable and become predictable as the array gets sorted.    */
#include "common.h"
int main(int argc, char **argv) {
    long n = arg_or(argc, argv, 800);
    uint32_t *a = malloc(n * sizeof(uint32_t));
    if (!a) return 1;
    uint64_t seed = 777;
    for (long i = 0; i < n; i++) a[i] = (uint32_t)lcg_next(&seed);
    for (long i = 0; i < n - 1; i++)
        for (long j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) { uint32_t tmp = a[j]; a[j] = a[j + 1]; a[j + 1] = tmp; }
    uint64_t chk = 0;
    for (long i = 0; i < n; i++) chk = chk * 31 + a[i];
    printf("bsort: n=%ld sorted=%d checksum=%016llx\n", n, a[0] <= a[n - 1],
        (unsigned long long)chk);
    free(a);
    return 0;
}

```